

Sistemas Operativos

Resumen teorías

Clase 1 - Kernel - Parte 1	6
Sistema Operativo	6
Características	6
Perspectiva desde el usuario	6
Perspectiva desde la administración de recursos	6
Objetivos del SO	7
Componentes del SO	7
Kernel	8
En Linux ¿Qué es?	8
Apoyo del hardware	9
Modos de ejecución	9
Bit de Modo	9
Tipos de kernel de acuerdo a su arquitectura de desarrollo	10
Monolítico	10
Microkernel	11
Híbrido	11
ExoKernel (Kernel de Exoesqueleto)	13
Kernel de Tiempo Real (Real-Time Kernel)	13
Clase 2 - Kernel - Parte 2	14
Kernel de Linux	14
Organización/Subsistemas del código fuente del Kernel de Linux	14
Clase 3 - System calls	16
System call	16
Ejemplo: read	16
Ejemplo: write	19
Registros usados	19
Categorías de system calls	19
En GNU/Linux	21
Clase 4 - Hilos	23
Proceso	23
Evolución del hardware	24
Sistemas Dual-processor (DP)	24
Sistemas Dual-core	24
Multithreading Simultáneo	25
Evolución del Software	25
Evolución de los sistemas operativos	25
SO Multithreading vs. Multithreading	26
SO actuales	26
Procesos e hilos	26

Ventajas de los hilos	27
Estructura de un hilo	28
Procesos vs hilos	29
Estados de un hilo	30
Tipos de threads	30
User Level Thread (ULT)	30
Ventajas	31
Desventajas	31
Kernel Level Thread (KLT)	31
Ventajas	32
Desventajas	32
Combinaciones	32
Uno a uno	32
Muchos a uno	33
Muchos a muchos	33
Clase 5 - Virtualización	35
Tipos de virtualización	35
¿Por qué virtualizar?	36
Virtualización	36
Monitor de Máquinas Virtuales (VMM)	37
Características de una VM (Popek y Goldberg)	37
Instrucciones privilegiadas	38
Anillos de privilegios en x86	39
Trap and emulate	40
Hipervisores	40
Hipervisor tipo 1	41
Hipervisor tipo 2	41
Comparación	42
Técnicas de virtualización	42
Emulación	42
Full virtualization	43
Traducción binaria	43
Asistida por Hardware	43
Paravirtualización	43
Implementaciones	44
Virtualización vs paravirtualización	45
Clase 6 - cgroups, namespaces y containers	46
chroot	46
cgroups	46
v1 vs v2	47
cgroups v1	48
cgroups v2	50
Namespaces	53
PID namespace	54

Mount namespace	55
User namespace	55
Containers	56
Clase 7 - Docker	59
Containers vs. VMs	59
¿Qué es Docker?	60
¿Cómo funciona?	60
Union Mount Filesystems	60
Definiciones	61
Imágenes y capas	61
DockerFile	62
Contenedores	63
Almacenamiento (Storage)	63
Networking	64
Clase 8 - Docker compose y Podman	65
Docker compose	65
Podman	66
Pod	67
Estandarización	68
Orquestador	69
Clase 9 - Seguridad	70
Protección ≠ seguridad	70
Seguridad	70
Protección	71
Políticas ≠ mecanismos	71
Objetos y dominios	71
Dominios y procesos	72
Matriz de acceso	73
Operación switch	74
Operación copy	75
Variantes	75
Operación owner	75
Operación control	76
Resumen	77
Implementación	77
Tabla global	77
Lista de control de acceso (ACL)	78
Lista de capacidades	79
Sistemas confiables	79
Explotación de errores en código	80
Clase 10 - Multiprocesadores	81
Origen	81
Esquemas	81
Multiprocesadores con memoria compartida	81

Multicomputadora con memoria independiente/pasaje de mensajes	82
Sistemas distribuidos	83
Chips multinúcleo	83
Tipos de SO multiprocesador	84
Responsabilidades del Sistemas Operativo	84
Cada CPU con su SO (modelo poco utilizado)	85
Maestro - Esclavo	85
SMP - Multiprocesadores Simétricos	86
Sincronización de multiprocesadores	87
Planificación de multiprocesadores	88
Planificación de hilos KLT independientes	88
Aprovechamiento de la caché	89
Planificación de hilos que trabajan en conjunto	89
Planificación por pandillas (una solución al problema anterior)	90
Multicomputadoras	91
Software de comunicación a nivel de usuario	92
Pasaje de mensajes	92
RPC	92
Planificación	92
Balanceo de carga	92
Sistemas distribuidos	93
Middleware	94
Clase 11 - Deadlocks	95
Definición	95
Ejemplos	95
Recursos	95
Representación de procesos y recursos	96
Hay deadlock si...	99
Condiciones para deadlock	99
Métodos para el tratamiento de deadlock	99
Prevention vs avoidance	100
Prevenir	100
Exclusión mutua	100
Retención y espera	100
No apropiación	101
Espera circular	101
Evitar	102
Estado seguro e inseguro	102
Algoritmos para evitar el deadlock	103
Algoritmo del banquero	104
Detección y recuperación	106
Detección	106
Instancia única	106
¿Cuándo uso el algoritmo de detección?	107

Recuperación	107
Matar procesos	107
¿En qué orden debemos optar por abortar/terminar?	108
Resumen de detección y recuperación	109
Estrategia combinada para el manejo de deadlocks	109

Clase 1 - Kernel - Parte 1

Sistema Operativo

Un sistema operativo es un software que actúa como intermediario entre el usuario de una computadora y su hardware

Es software por lo que necesita procesador y memoria para ejecutar

Características

- Gestiona el HW
- Controla la ejecución de los procesos
- Interfaz entre aplicaciones y HW
- Actúa como intermediario entre un usuario de una computadora y el HW de la misma

Perspectiva desde el usuario

- Abstracción con respecto a la arquitectura:
 - Arquitectura: conjunto de instrucciones, organización de memoria, E/S, estructura de bus)
- El SO “oculta” el HW y presenta a los programas abstracciones más simples de manejar
- Los programas de aplicación son los “clientes” del SO
- Comparación: uso de escritorio y uso de comandos de texto
- Comodidad, “amigabilidad” (friendliness)

Perspectiva desde la administración de recursos

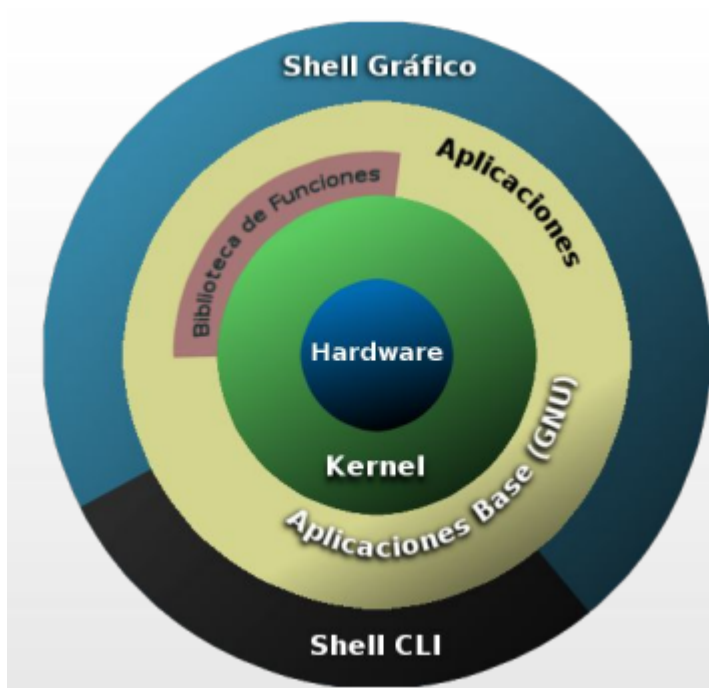
- Administra los recursos de HW de uno o más procesos
- Provee un conjunto de servicios a los usuarios del sistema
- Maneja la memoria secundaria y dispositivos de I/O (Input/Output – Entrada/Salida)
- Ejecución simultánea de procesos
- Multiplexación en tiempo (CPU) y en espacio (memoria)

Objetivos del SO

- Comodidad:
 - Hacer más fácil el uso del hardware (PC, servidor, switch, router, controlador específico)
- Eficiencia:
 - Hacer un uso más eficiente de los recursos del sistema
- Evolución:
 - Permitir la introducción de nuevas funciones al sistema sin interferir con funciones anteriores

Componentes del SO

- Kernel
- Shell (GUI/CLI)
- Herramientas:
 - Editores
 - Compiladores
 - Librerías



Kernel

Es una “porción de código”:

- Se encuentra en memoria principal
- Se encarga de la administración de los recursos

Implementa servicios esenciales:

- Manejo de memoria
- Manejo de la CPU
- Administración de procesos
- Comunicación y Concurrencia
- Gestión de la E/S

En Linux ¿Qué es?

- Programa que ejecuta programas y gestiona dispositivos de hardware
- Encargado de que el software y el hardware puedan trabajar juntos
- Principales funciones:
 - Administración de memoria principal
 - Administración de uso de la CPU
- Es de código abierto a los usuarios (kernel/sched.c)
- En una misma estructura de código fuente se da soporte a todas las arquitecturas
- Liberado bajo licencia GPLv2
- En un sentido estricto es el Sistema Operativo
- Es responsable de facilitar a los procesos acceso seguro al hardware
- Para ello, utiliza una interfaz conocida como system calls
- Los procesos de usuario son clientes del Sistema Operativo
- El Kernel gestiona y atiende los requerimientos de los distintos procesos siguiendo un criterio de “equidad”
- El Kernel se ejecuta en modo supervisor o privilegiado
- En este modo se tiene acceso al conjunto completo de instrucciones que permiten, entre otras cosas:
 - Acceder al hardware
 - Direccionar la memoria
 - Programar la CPU
- Los procesos se ejecutan en modo usuario

- Cuando un proceso requiere acceso al hardware, lo hace a través de una llamada al sistema operativo □ “System Call”
- La capacidad de ejecutar código en modo supervisor o modo usuario, es provista por el hardware que trabaja en conjunto con el Sistema Operativo

Apoyo del hardware

Se ve en los siguientes aspectos:

- Modos de ejecución: define limitaciones en el conjunto de instrucciones que se puede ejecutar en cada modo
- Interrupción de clock: se debe evitar que un proceso se apropie de la CPU
- Protección de la memoria: se deben definir límites de memoria a los que puede acceder cada proceso (registros base y límite)

Modos de ejecución

El bit en la CPU indica el modo actual

Las instrucciones privilegiadas deben ejecutarse en modo Supervisor o Kernel, ya que necesitan acceder a estructuras del kernel, o ejecutar código que no es del proceso

En modo Usuario, el proceso puede acceder sólo a su espacio de direcciones, es decir a las direcciones “propias”

Bit de Modo

Cuando se arranca el sistema, arranca con el bit en modo supervisor

Cada vez que comienza a ejecutarse un proceso de usuario, este bit se DEBE PONER en modo usuario (mediante una interrupción)

Cuando hay un trap o una interrupción, el bit de modo se pone en modo Kernel

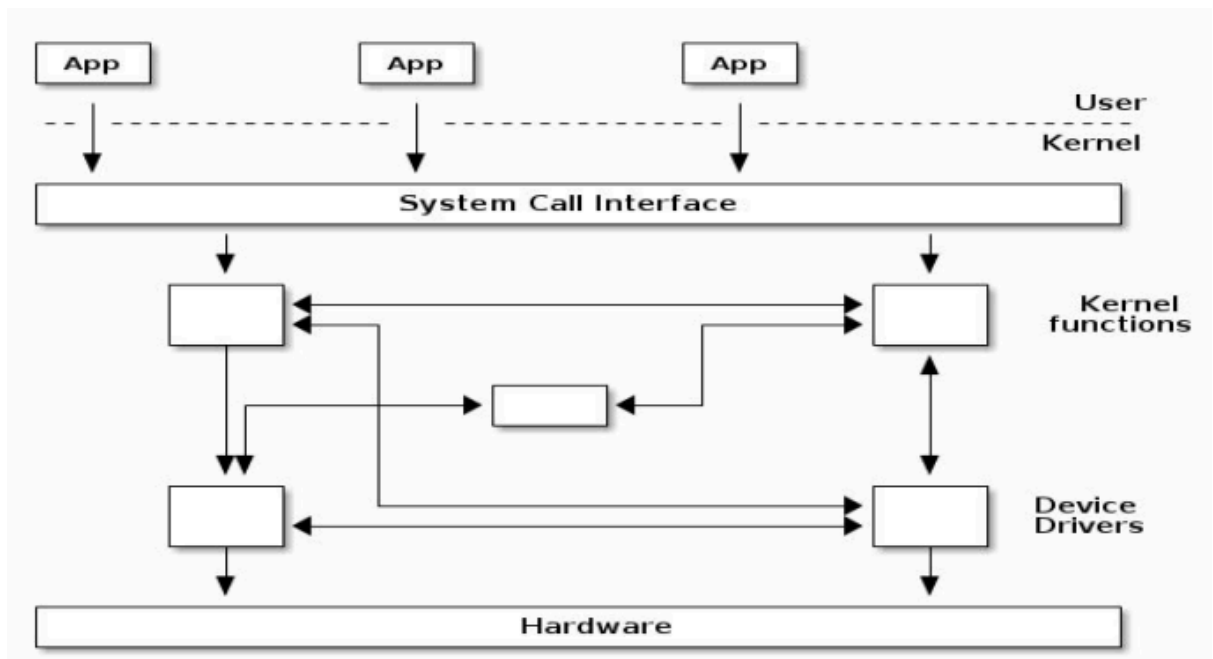
Es la única forma de pasar a Modo Kernel

No es el proceso de usuario quien hace el cambio explícitamente

Tipos de kernel de acuerdo a su arquitectura de desarrollo

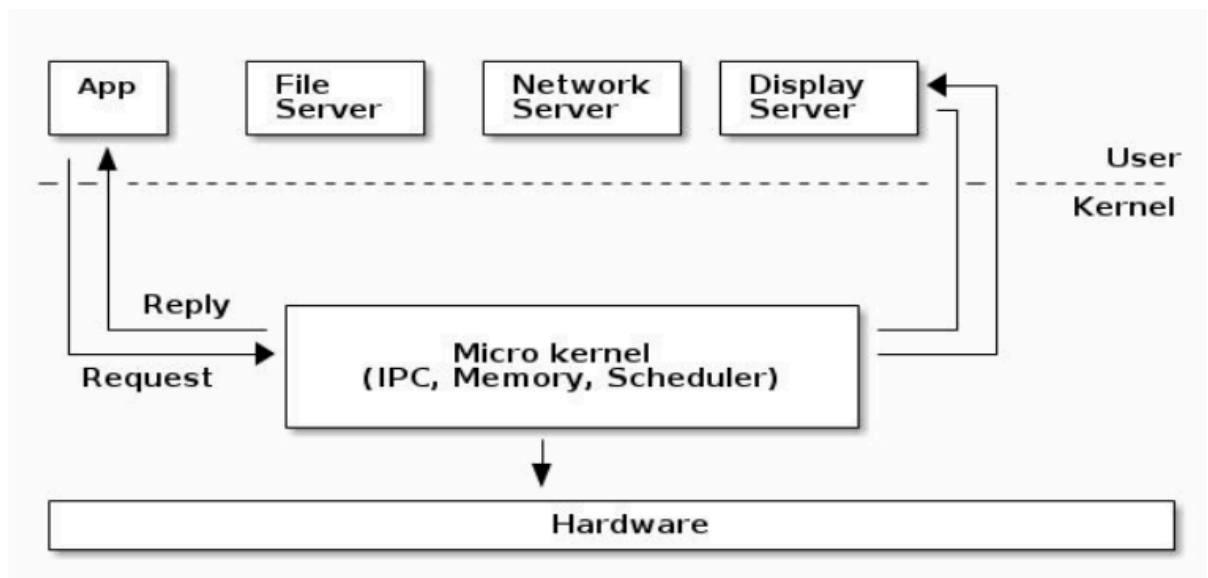
Monolítico

- Incluye todos los servicios del sistema operativo en un solo bloque de código que se ejecuta en modo supervisor (gestión de procesos, memoria, archivos, controladores, etc.)
- Posee distintos subsistemas y la funcionalidad de cada uno es accedida directamente desde otro a través de funciones públicas
- Gestión completa de recursos (memoria, cpu, E/S)
- Posee acceso completo a los recursos de hardware □ Eficiencia al no requerir cambios de modo mientras se ejecuta
- Manejo directo de interrupciones y excepciones
- Permite opcionalmente un enfoque modular, pero los módulos siguen ejecutándose en modo Kernel □ GNU/Linux
- Al ser un único bloque de código, es más complejo de manejar, administrar y mantener
- Ejemplos: Linux, Unix, FreeBSD



Microkernel

- Minimiza la cantidad de código que se ejecuta en modo supervisor con el fin de hacerlo más liviano respecto a un monolítico
- Incluye funciones más esenciales, como la gestión de procesos, comunicación entre procesos y gestión de memoria
- Otros servicios (como controladores de hardware, sistemas de archivos y red) se ejecutan en espacio de usuario
- Es altamente modular permitiendo su personalización a través de la adición o eliminación de módulos de funcionalidad
- Dado que la mayoría de los servicios se ejecutan en modo usuario, suele ser más estable y seguro que los monolíticos
- Provee un rendimiento inferior por los cambios de modo constantes que requiere para su ejecución
- Su desarrollo y mantenimiento es más eficiente dado a la separación de funciones
- Ejemplos: Minix, QNX y L4



Híbrido

- Combina características de los Kernels monolíticos y Microkernels
- Tiene un núcleo más pequeño que un monolítico. Incluye algunos servicios en modo kernel (eficiencia) y otros se ejecutan en modo usuario (seguridad)
- Son modulares. Ofrecen un equilibrio entre rendimiento y modularidad lo que permite actualizaciones más sencillas y una mayor flexibilidad y facilidad a su desarrollo

- Ofrecen un mejor rendimiento que los microkernels y mayor modularidad/seguridad que los monolíticos
- Suelen ser una alternativa equilibrada y atractiva
- Ejemplos: Windows y MacOS

Característica	Monolítico	Microkernel	Híbrido
Código en modo supervisor	Todo el kernel (gestión de procesos, memoria, E/S, drivers) corre en modo kernel.	Solo lo esencial (IPC, scheduling, gestión memoria) en kernel; resto en espacio usuario.	Núcleo reducido en kernel; servicios adicionales en espacio usuario.
Modularidad	Opcional (módulos cargables) pero siguen en kernel.	Muy alta: controladores y servicios en user-space.	Alta: mezcla de módulos en kernel y servicios en user-space.
Rendimiento	Muy alto (mínimos cambios de contexto).	Menor (frecuentes cambios de contexto entre modos).	Equilibrado: mejor que microkernel, cercano al monolítico.
Seguridad/Estabilidad	Menor que microkernel (errores en un driver pueden afectar todo).	Alta (fallos quedan confinados en user-space).	Intermedio: mejor aislamiento que monolítico, menos overhead que microkernel.
Gestión de interrupciones/excepciones	Directa en kernel.	El Kernel maneja lo mínimo; notifica a user-space.	Kernel maneja interrupciones; algunos servicios delegados.
Complejidad de desarrollo/mantenimiento	Alta (gran base de código en un solo espacio).	Moderada-baja (código crítico muy reducido).	Media: combinación de ambos modelos.

Ejemplos	Linux, FreeBSD	Unix, Minix, QNX, L4	Windows NT, macOS (XNU)
----------	-------------------	-------------------------	----------------------------

ExoKernel (Kernel de Exoesqueleto)

- Provee servicios muy básicos al sistema operativo, evitando alto nivel de abstracción (como hilos o archivos)
- Ofrece acceso directo a recursos del hardware, permitiendo a los desarrolladores construir soluciones personalizadas
- Provee primitivas básicas para la gestión de memoria y deja las políticas de gestión de memoria a bibliotecas en espacio de usuario
- Ofrece primitivas para que las aplicaciones gestionen sus propios procesos
- Los controladores se ejecutan en espacio de usuario
- La mínima cantidad de código reduce los vectores de ataque y los errores, lo que hace que los exokernels sean ideales para entornos especializados como sistemas en tiempo real o embebidos

Kernel de Tiempo Real (Real-Time Kernel)

- Debe garantizar que las operaciones se completen dentro de marcos de tiempo conocidos y predecibles
- Utiliza algoritmos de scheduling de CPU avanzados para asegurar que las tareas se ejecuten en el orden correcto y cumplir con los plazos establecidos
- Implementa mecanismos de asignación de memoria estática o dinámica optimizados para prevenir retrasos indeseados, evitando deseablemente los fallos de páginas
- Soporta multitasking apropiativo donde las tareas de alta prioridad suspenden a las de menor
- Provee mecanismos de seguridad robustos
- Ejemplos: VxWo y QNX

Clase 2 - Kernel - Parte 2

Kernel de Linux

Escrito en:

- C mayoritariamente
- Assembler: para instrucciones especiales y de bajo nivel
- Rust: para implementar módulos (incluido recientemente a partir de v6.1)

El Kernel de GNU/Linux está dividido en subsistemas

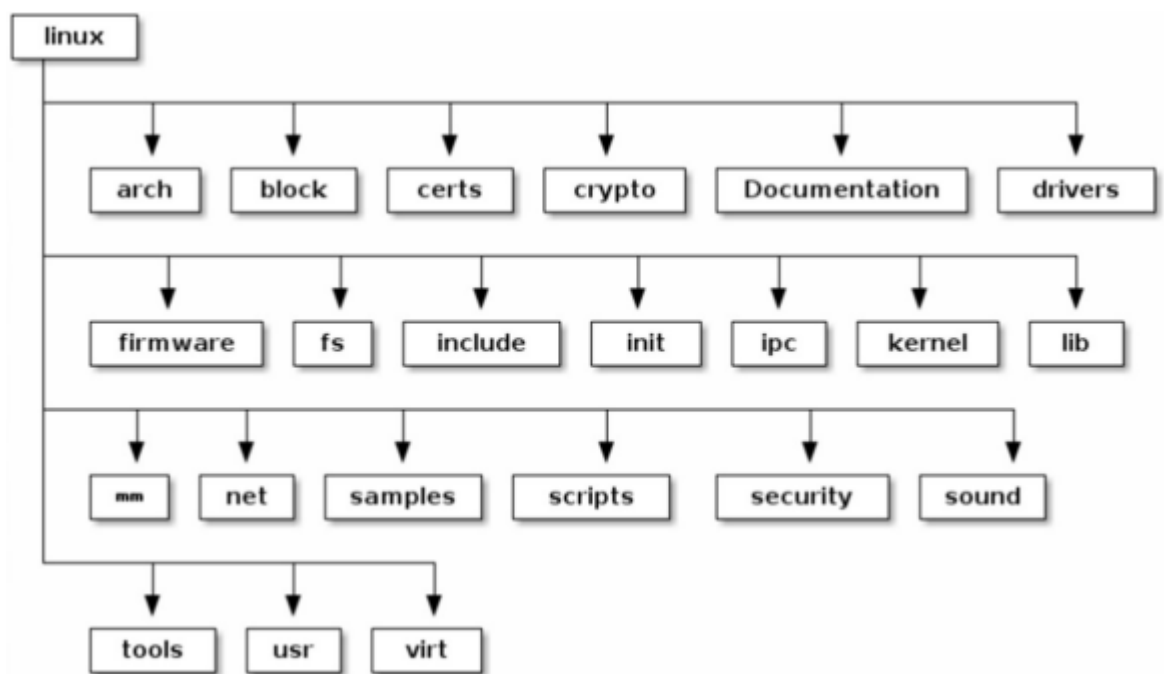
Cada subsistema es mantenido por uno o más responsables

El responsable de cada subsistema acepta o no parches o pull requests de los desarrolladores

Luego el responsable interactúa con Linus para incluir las modificaciones en una versión candidata "rc"

Cada responsable (maintainer) tiene su propio tree para el desarrollo

Organización/Subsistemas del código fuente del Kernel de Linux



Es un núcleo monolítico híbrido:

- Los drivers y el código del Kernel se ejecutan en modo privilegiado

- Lo que lo hace híbrido es la posibilidad de cargar y descargar funcionalidad a través de módulos

Clase 3 - System calls

System call

Es el mecanismo utilizado por un proceso de usuario para solicitar un servicio al SO

Recordemos:

- Un proceso de usuario es una porción de código básico que ejecuta instrucciones de código propio en un entorno controlado y limitado de ejecución (sumas, restas, comprobaciones, etc.)
- Está limitado a su propio espacio de direcciones y no posee acceso directo al hardware

El responsable de garantizar que los procesos de usuario puedan ejecutarse controladamente y sin interferirse, es el SO, que se ejecuta en modo kernel

¿Cómo logra un proceso de usuario acceder a funciones o servicios que deben ser protegidos por el OS?

La respuesta es sencilla: system calls

Una llamada al sistema es un método que puede invocar un proceso para solicitar cierto servicio al SO

El SO provee una interfaz de programación (API) que los procesos pueden utilizar en cualquier momento para solicitar recursos gestionados por él mismo

El SO actúa como un servidor que recibe continuamente requerimientos de sus clientes (procesos) y determina cuál es el mejor mecanismo de satisfacerlos

Para poder llevar adelante la tarea, define un mecanismo y los parámetros que debe enviar el programador para solicitar los servicios

Del mismo modo define los valores de retorno

En Unix, GNU/Linux la API que define estos mecanismos se llama libc

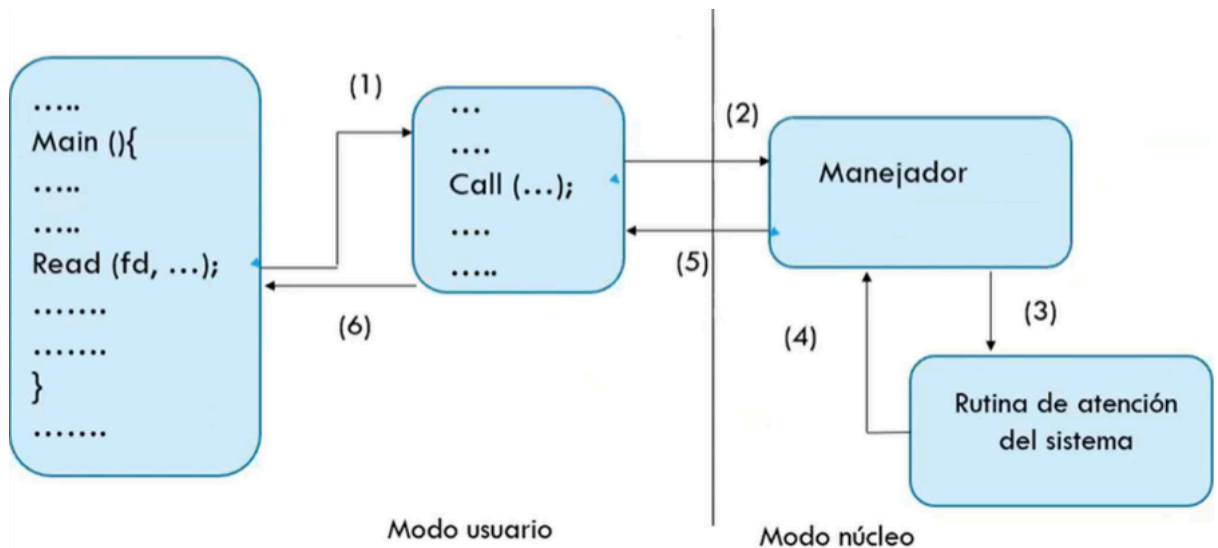
Ejemplo: read

Cuando un proceso necesita leer un archivo, requiere acceder al hardware

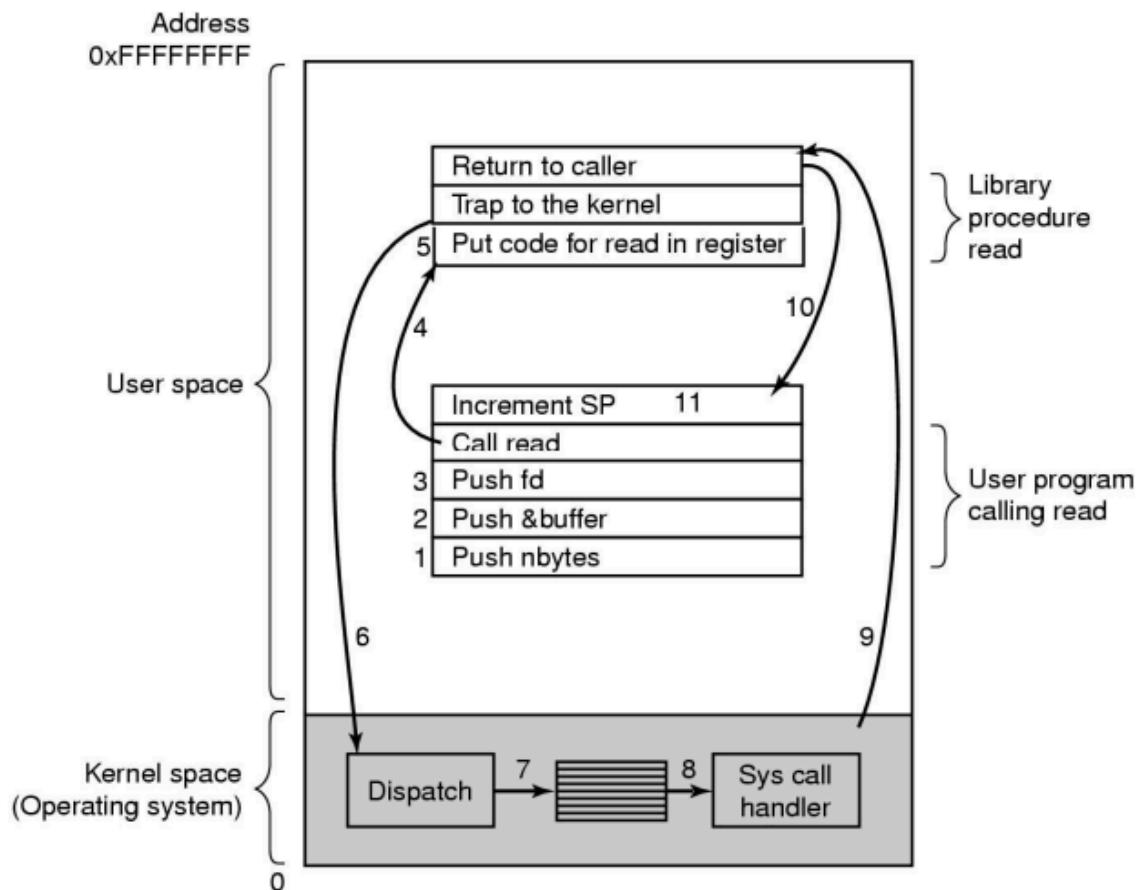
Este acceso solo puede realizarlo el SO, con lo cual expone un servicio que permite que sea invocado por el proceso:

count=read(file, buffer, nbytes);

- file: archivo que se quiere leer
- buffer: área de memoria en la que será leído
- nbytes: cantidad de bytes a leer



Las llamadas al sistema se llevan a cabo en una serie de pasos:



1. El primer paso para poder invocar a read (implementado en la librería de usuario), es meter en la pila los parámetros necesarios
2. Se invoca a la función read implementada en la librería y se comienza su ejecución
3. Read será una función sencilla que básicamente es la que indicará el número de system call que se debe ejecutar y permitirá realizar la llamada al sistema correspondiente
4. La función read será la encargada de ejecutar el TRAP (interrupción por software) para cambiar de modo usuario a modo kernel y pasar el control al SO
5. Para todas las system calls se utiliza la misma interrupción. La forma de identificar la Syscall invocada es a través del valor del registro
6. Una vez que el SO tiene el control, verificará cuál es la llamada al sistema que debe atender y ejecutará el código correspondiente
7. En este punto se accede al dispositivo de almacenamiento para obtener el archivo solicitado y leerlo en memoria

Ejemplo: write

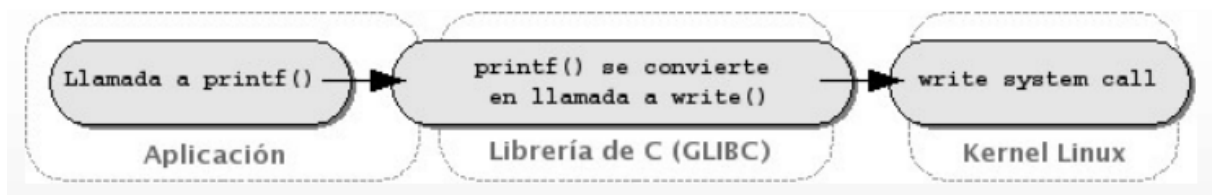
El código de la librería realiza las siguientes acciones

Para activar la system call se indica:

- El número de syscall que se quiere ejecutar
- Los parámetros de esa syscall

Luego se emite un aviso al SO (trap) para pasar a modo Kernel y gestionar la system call

Se evalúa la system call deseada y se ejecuta



Registros usados

Registros x86 en 32 bits

- EAX lleva el numero de syscall que se desea ejecutar
- EBX lleva el primer parámetro
- ECX lleva el segundo parámetro
- EDX ...
- ESI
- EDI

Instrucción que inicia la system call: `int 80h` (32 bits, syscall en 64 bits)

Categorías de system calls

- Control de procesos
- Manejo de archivos
- Manejo de dispositivos
- Mantenimiento de información del sistema
- Comunicaciones

Process management

Call	Description
<code>pid = fork()</code>	Create a child process identical to the parent
<code>pid = waitpid(pid, &statloc, options)</code>	Wait for a child to terminate
<code>s = execve(name, argv, environp)</code>	Replace a process' core image
<code>exit(status)</code>	Terminate process execution and return status

File management

Call	Description
<code>fd = open(file, how, ...)</code>	Open a file for reading, writing or both
<code>s = close(fd)</code>	Close an open file
<code>n = read(fd, buffer, nbytes)</code>	Read data from a file into a buffer
<code>n = write(fd, buffer, nbytes)</code>	Write data from a buffer into a file
<code>position = lseek(fd, offset, whence)</code>	Move the file pointer
<code>s = stat(name, &buf)</code>	Get a file's status information

Directory and file system management

Call	Description
<code>s = mkdir(name, mode)</code>	Create a new directory
<code>s = rmdir(name)</code>	Remove an empty directory
<code>s = link(name1, name2)</code>	Create a new entry, name2, pointing to name1
<code>s = unlink(name)</code>	Remove a directory entry
<code>s = mount(special, name, flag)</code>	Mount a file system
<code>s = umount(special)</code>	Unmount a file system

Miscellaneous

Call	Description
<code>s = chdir(dirname)</code>	Change the working directory
<code>s = chmod(name, mode)</code>	Change a file's protection bits
<code>s = kill(pid, signal)</code>	Send a signal to a process
<code>seconds = time(&seconds)</code>	Get the elapsed time since Jan. 1, 1970

UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time

En GNU/Linux

En GNU/Linux, las System Calls son identificadas por un número

Pueden tener como máximo 6 (seis) parámetros

Para x86 en 32 bits está definida en arch/x86/entry/syscalls/syscall_32.tbl

Para x86 en 64 bits está definida en arch/x86/entry/syscalls/syscall_64.tbl

La primer tarea que realiza el dispatcher cuando se produce una interrupción es verificar el número en la tabla correspondiente y ejecutar las funciones asociadas

Los parámetros de la System Call deben manejarse con cuidado, dado que se configuran en el espacio de usuario, por lo que no se puede asumir que sean correctos

En el caso de pasarse punteros, no pueden apuntar al espacio del Kernel por cuestiones de seguridad

De no verificarse, en un read por ejemplo el buffer podría tener una dirección del Kernel y sobrescribir datos sensibles

Los punteros deben ser siempre válidos

De no verificarse podría producir Kernel Panic

El Kernel deberá tener acceso al espacio de usuario con APIs especiales que garanticen que se accede al espacio de direcciones de quien invocó la System Call (get_user(), put_user(), copy_from_user(), copy_to_user())

En UNIX la API principal para la invocación de la System Call es libc:

- Es la API principal del SO
- Provee las librerías estándar de C
- Es una interfaz entre aplicaciones de usuario y las System Calls (System Call Wrappers)

La funcionalidad de la libc y las System Calls están definidas por el estándar POSIX

Busca proveer una interfaz común para lograr portabilidad

Busca que la interacción sea con la API y no con el Kernel

Clase 4 - Hilos

Proceso

Es la unidad básica de utilización de CPU

Es un programa en Ejecución

Unidad de asignación de los recursos

Conceptos relacionados con proceso:

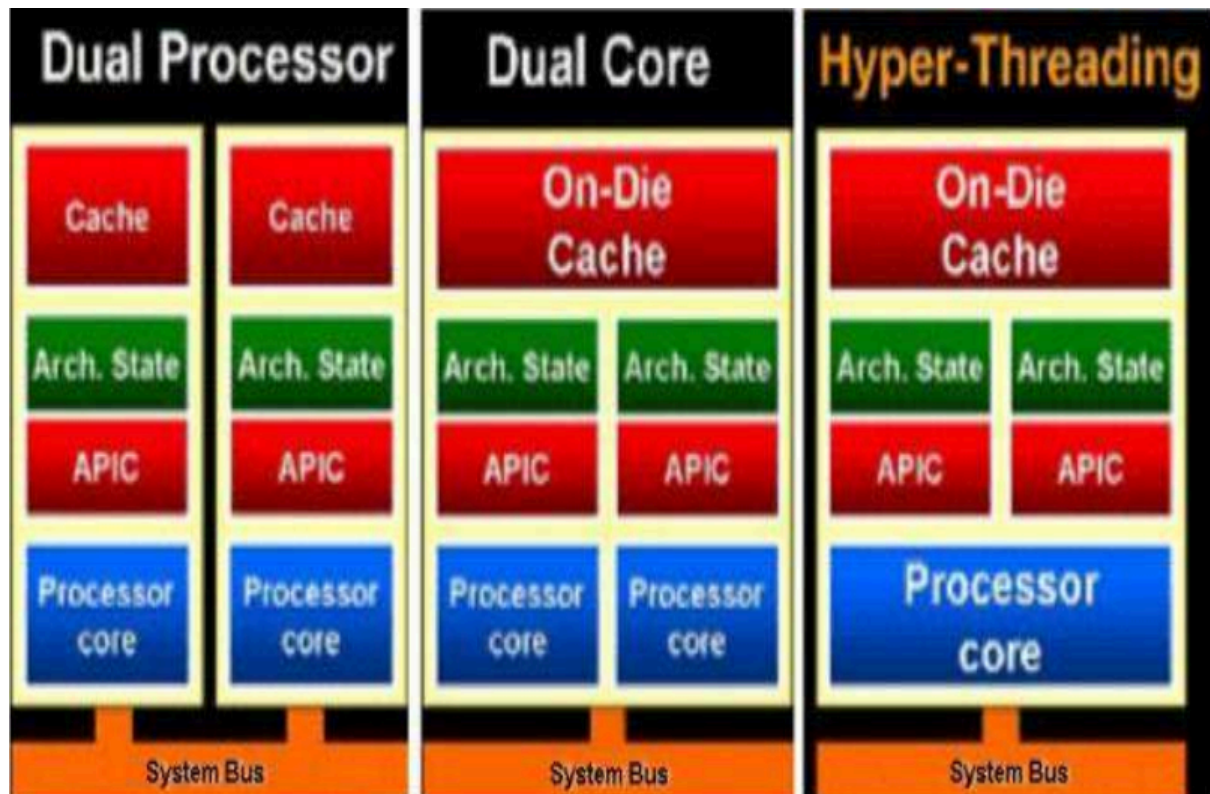
- Espacio de direcciones
- Punteros a los recursos asignados (stacks, archivos, etc.)
- Estructuras asociadas: PCB, tablas

En los primeros sistemas operativos, cada proceso tenía un espacio de direcciones y un solo hilo de control:

- Un único flujo secuencial de ejecución
- Se ejecuta una instrucción y cuando finaliza se ejecuta la siguiente

Para ejecutar otro proceso, se debe llevar adelante un cambio de contexto

Evolución del hardware



Sistemas Dual-processor (DP)

- Tiene 2 procesadores físicos en el mismo chasis
- Pueden estar en la misma motherboard o no
- Cache y controlador independientes

Sistemas Dual-core

- Una CPU con dos cores por procesador físico
- Un circuito integrado tiene 2 procesadores completos
- Los 2 procesadores combinan cache y controlador

En ambos casos, las APIC (Advanced Programmable Interrupt Controllers) están separadas por procesador. De esta manera proveen administración de interrupciones por procesador

Multithreading Simultáneo

Generalmente conocido como Hyper Threading, lo cual es propietario de las CPU Intel

Permite que el software programado para ejecutar múltiples hilos (multi-threaded) procese los hilos en paralelo dentro de un único procesador

La tecnología simula dos procesadores lógicos dentro de un único procesador físico

Duplica solo algunas “secciones” de un procesador:

- Registros de Control (MMU, Interrupciones, Estado, etc)
- Registros de Propósito General (AX, BX, PC, Stack, etc.)

Resultado: mejora en el uso del procesador (entre 20 y 30%)

Evolución del Software

Es común dividir un proceso en diferentes “tareas” que, independientemente o colaborativamente, solucionan el problema

Es común contar con un pool de procesadores para ejecutar nuestros procesos simultáneamente

Los lenguajes brindan herramientas que nos permiten separar las diferentes “tareas” de los programas en unidades de ejecución diferentes:

- Java – heredar de “Thread”, implementar la interface “Runnable”
- Delphi – Heredar de “TThread”
- Ruby – Thread.new{CODIGO}
- PHP – Heredar de Thread
- Javascript – HTML5 Web Workers

Evolución de los sistemas operativos

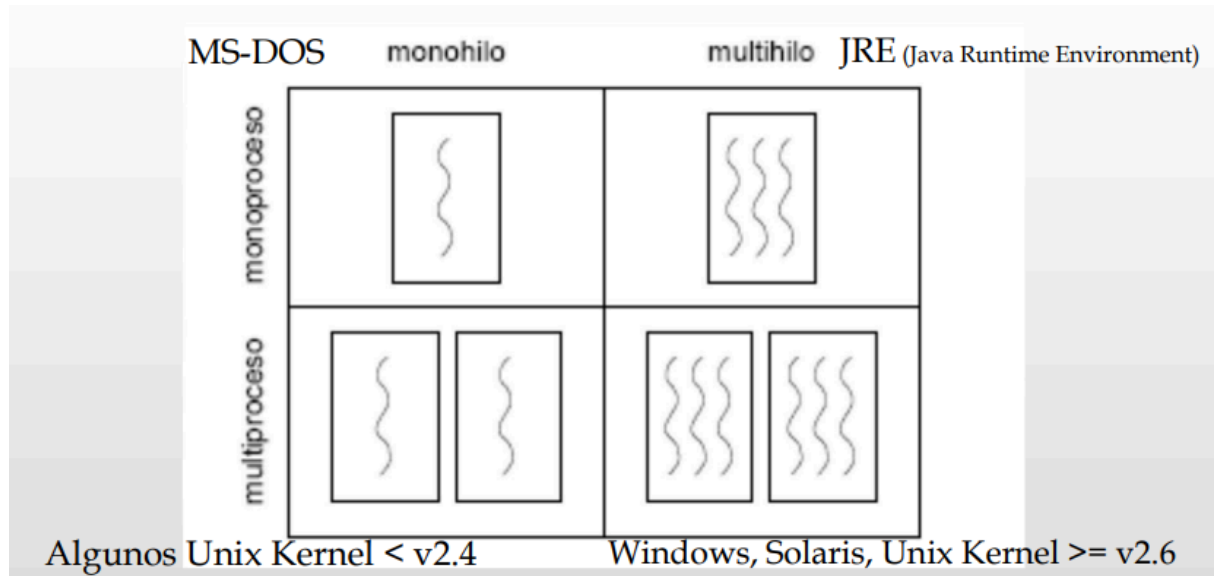
Hay situaciones en las que es conveniente tener varios hilos de control en el mismo espacio de direcciones:

- Ejecutarlos en cuasi-paralelo, como si fueran procesos (casi) separados
- Compartir el espacio de direcciones

Unidad básica de utilización de CPU: hilo

Unidad básica de asignación de RECURSOS: proceso

SO Monothreading vs. Multithreading



SO actuales

Proceso:

- Espacio de direcciones
- Unidad de propiedad de recursos
- Conjunto de threads (eventualmente uno)

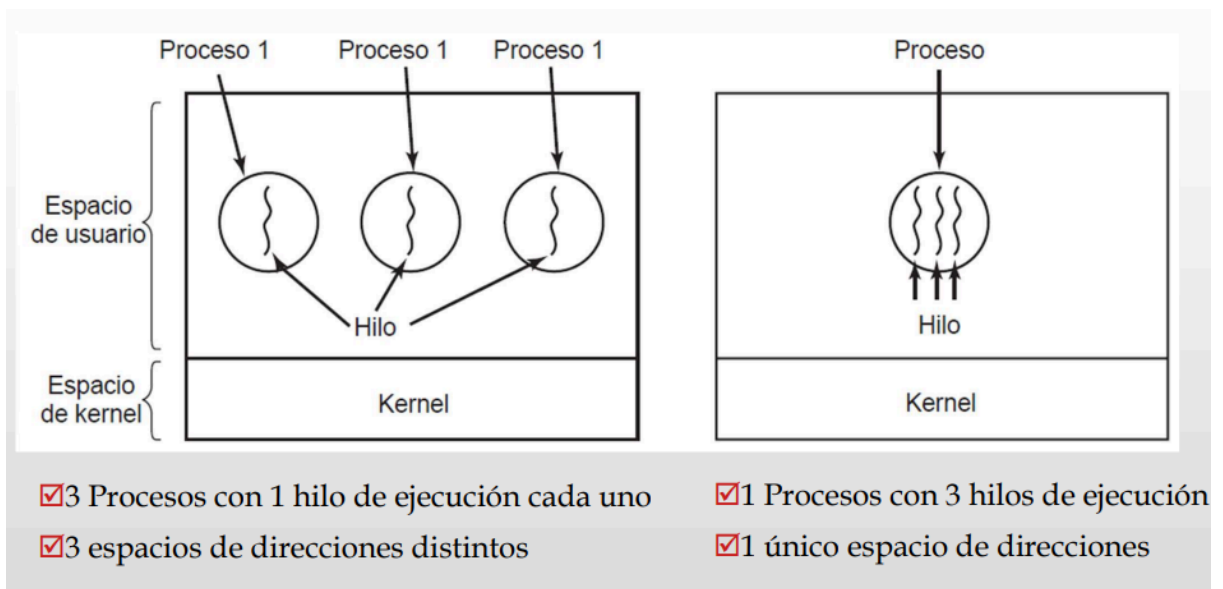
Thread:

- Unidad de trabajo (hilo de ejecución)
- Contexto del procesador
- Stacks de Usuario y Kernel
- Variables propias
- Acceso a la memoria y recursos del proceso

Procesos e hilos

¿Por qué dividir una aplicación en threads?

- Respuestas percibidas por los usuarios, paralelismo/ejecución en background
 - Ejemplo: El servicio de impresión de un editor de texto se ejecuta en background y nos permite seguir editando
- Aprovechar las ventajas de múltiples procesadores:
 - Con n CPUs pueden ejecutarse n threads al mismo tiempo
 - Pregunta: ¿Dada una aplicación con un único thread, agregar un nuevo procesador hará que esta se ejecute más rápido? (NO)
- Características complejas:
 - Sincronización
 - Escalabilidad: cantidad de threads por proceso? (# de CPU, excesivos cambios de contexto de hilos del mismo proceso...)



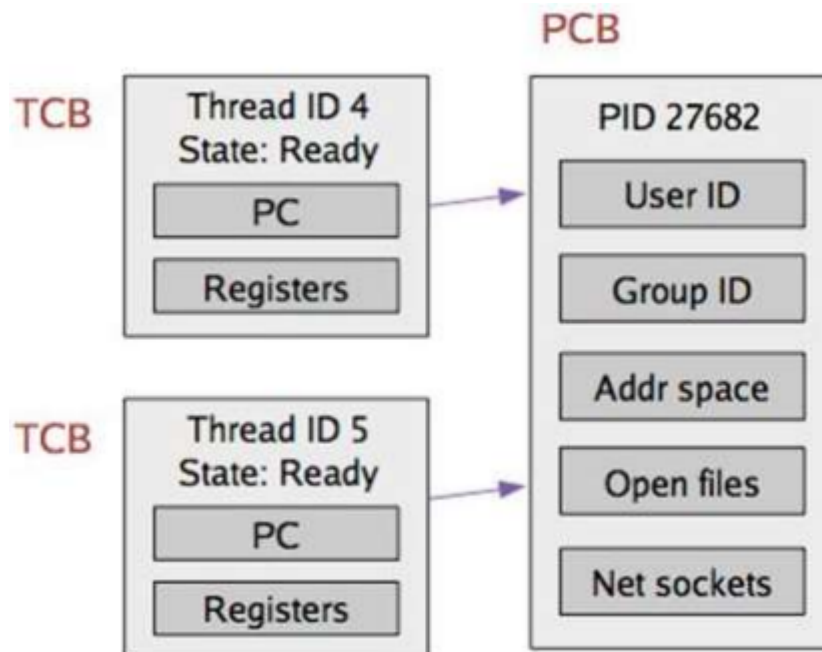
Ventajas de los hilos

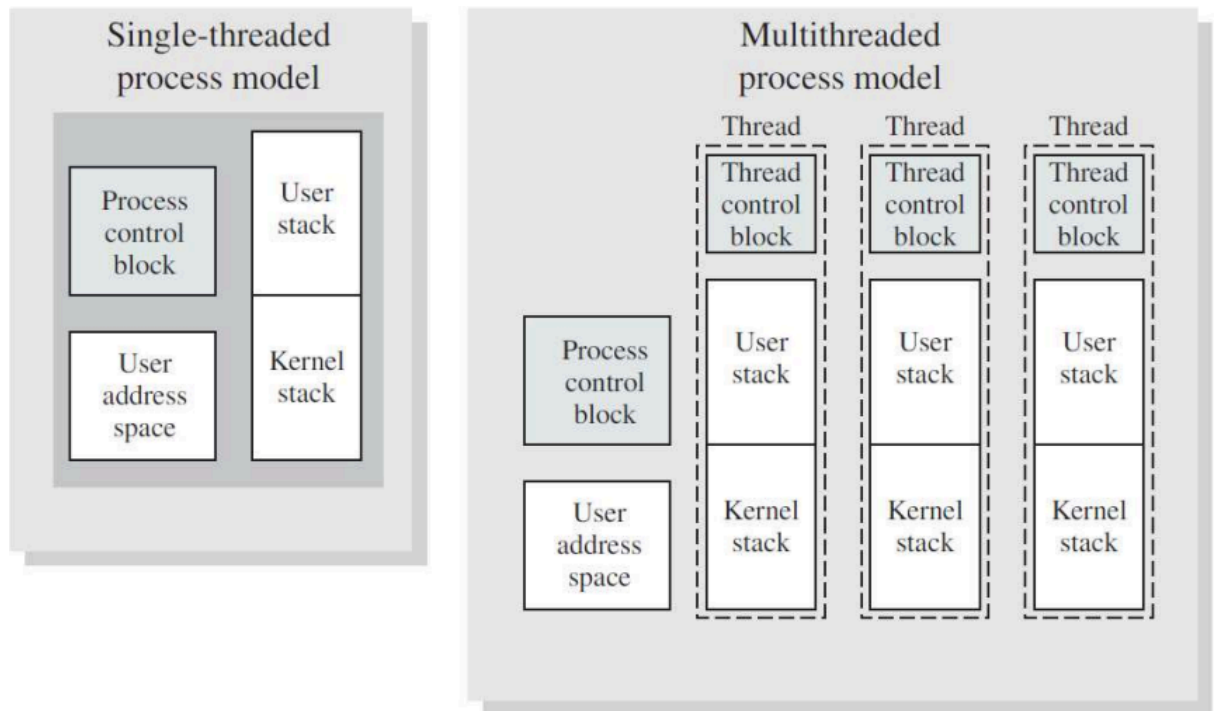
- Sincronización de Procesos
- Mejorar tiempos de Respuesta
- Compartir Recursos
- Economía

Estructura de un hilo

Cada hilo cuenta con:

- Un estado de ejecución
- Un contexto de procesador
- Stacks (uno en modo usuario y otro en modo kernel)
- Acceso a memoria y recursos del proceso:
 - Archivos abiertos
 - Señales
 - Código
 - Todos estos datos se compartirán con el resto de los hilos del proceso
- TCB – Thread Control Block





Procesos vs hilos

El uso de hilos en reemplazo de los procesos posee ventajas:

- **Context switch:**
 - Procesos: El SO debe intervenir con el fin de salvar el ambiente del proceso saliente y recuperar el ambiente del nuevo
 - Hilos: El cambio de contexto solo se realiza a nivel de registros y no espacio de direcciones. Lo lleva a cabo el proceso sin necesidad de intervención del SO
- **Creación:**
 - Procesos: Implica la creación de un nuevo espacio de direcciones, PCB, PC, etc. Lo lleva a cabo el SO
 - Hilos: Implica la creación de una TCB, registros, PC y un espacio para el stack. Lo hace el mismo proceso sin intervención del SO
- **Destrucción:**
 - Procesos: El SO debe intervenir con el fin de salvar el ambiente del proceso saliente y eliminar su PCB
 - Hilos: La tarea se realiza dentro del proceso sin necesidad de intervención del SO
- **Planificación:**

- Procesos: Es llevada a cabo por el sistema operativo. El cambio implica cambios de contexto continuos
- Hilos: Es responsabilidad del desarrollador quien debe planificar sus hilos. Es menos costoso, pero puede traer desventajas aparejadas
- Protección:
 - Procesos: El SO garantiza la protección a través de distintos mecanismos de seguridad. La comunicación entre ellos implica el uso de técnicas más avanzadas
 - Hilos: La protección debe darse desde el lado del desarrollo. Todos los hilos comparten el mismo espacio de direcciones. Un hilo podría bloquear la ejecución de otros

Estados de un hilo

Estados Mínimos: Ejecución, Listo y Bloqueado

Los eventos sobre procesos afectarán a todos sus Threads



Tipos de threads

User Level Thread (ULT)

La aplicación, en modo usuario, se encarga de la gestión (por medio de una Biblioteca de Threads)

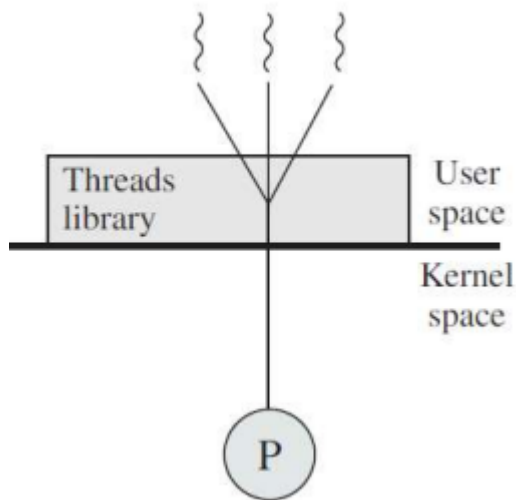
La Biblioteca deberá brindar funciones para: Crear, destruir, planificar, etc.

El Kernel “no se entera” de la existencia de Threads

Ejemplos:

- Java VM
- POSIX Threads

- Solaris Threads



Ventajas

- Intercambio entre hilos: comparten el espacio de direcciones
- Planificación independiente: cada proceso los planifica como más le conviene
- Podrían reemplazarse llamadas al sistema bloqueantes por otras que no bloqueen
- Portabilidad: pueden correr en distintas plataformas
- No requiere cambios para su "existencia"
- No es necesario que el SO soporte hilos

Desventajas

- No se puede ejecutar hilos del mismo proceso en distintos procesadores
- Si un hilo produce un Page Fault, todo el proceso se bloquea
- Un hilo podría monopolizar el uso de la CPU por parte del proceso
- Bloqueo del proceso durante una System Call bloqueante

Kernel Level Thread (KLT)

La gestión completa se realiza en modo Kernel

Ejemplos:

- Windows NT/2000
- Linux

Ventajas

Se puede multiplexar hilos del mismo proceso en diferentes procesadores
Independencia de bloqueos entre Threads de un mismo proceso

Desventajas

Cambios de modo de ejecución para la gestión (Planificación, creación, destrucción, etc.)

Combinaciones

Es posible combinar ULT y KLT

En este tipo de sistemas, la creación de hilos se realiza a nivel de usuario y los mismos son mapeados a una cantidad igual o menor de KLT

La sincronización de hilos en este modelo, permite que un hilo se bloquee y otros hilos del mismo proceso sigan ejecutándose

Permite que hilos de usuario mapeados a distintos KLT puedan ejecutarse en distintos procesadores

Este enfoque aprovecha las ventajas de ambos tipos

Threads: Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
M:N	Combines attributes of M:1 and 1:M cases.	TRIX

Uno a uno

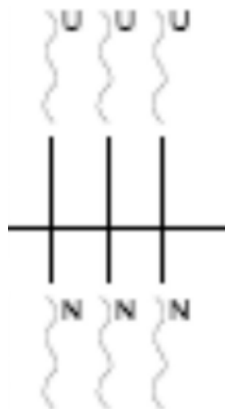
Cada ULT mapea con un KLT

Cuando se necesita un ULT se debe crear un KLT

Si se bloquea un ULT, otro hilo del mismo proceso puede seguir ejecutándose

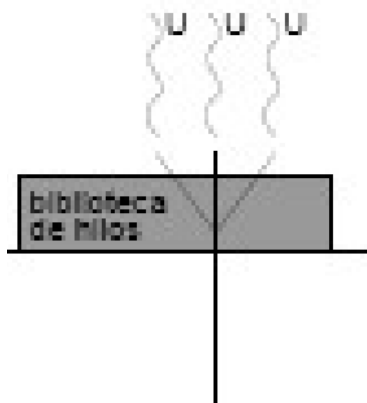
La concurrencia y/o paralelismo es máximo, ya que cada hilo puede correr en un procesador distinto

Introduce un costo alto, ya que cada vez que se crea un hilo de usuario se debe crear un KLT



Muchos a uno

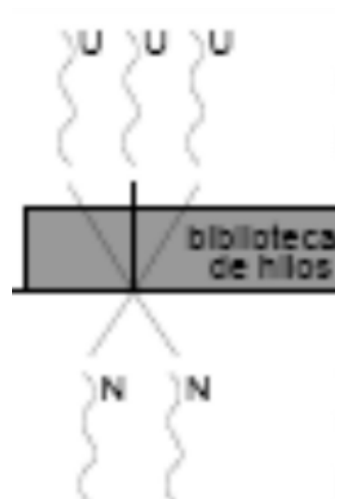
Muchos ULT mapean a un único KLT
Usado en sistemas que no soportan KLT
Si se bloquea un ULT, se bloquea el proceso
Java sobre un sistema que no soporta KLT



Muchos a muchos

Muchos ULT mapean a muchos KLT
Este modelo multiplexa los ULT en KLT, logrando un balance razonable:

- No tiene el costo del modelo 1:1
- Minimiza los problemas de bloqueo del modelo M:1



Clase 5 - Virtualización

Es una técnica que permite realizar “abstracción” de recursos de la computadora

Hoy en día también se virtualizan las redes, storages, data centers, etc...

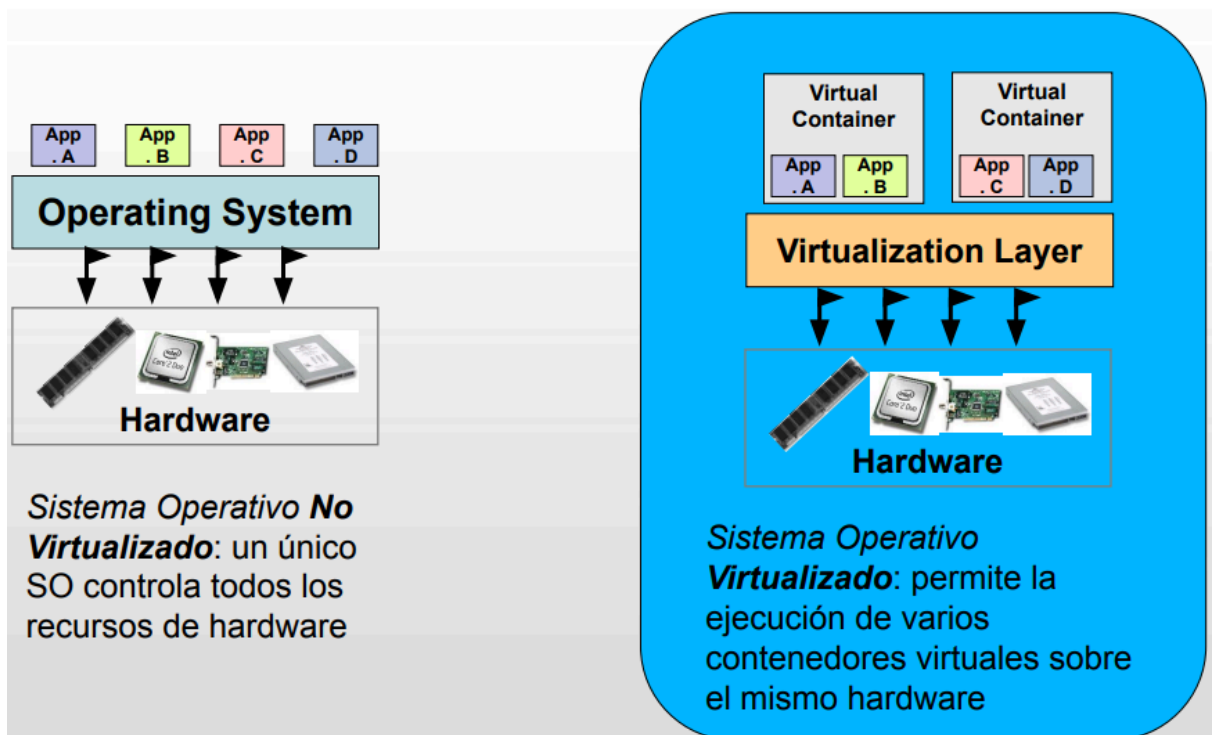
Es una capa abstracta que desacopla el hardware físico del sistema

Permite ocultar detalles técnicos a través de la encapsulación

Permite, entre otras cosas, que una computadora pueda realizar el trabajo de varias, a través de la compartición de recursos de un único dispositivo de hardware

Tipos de virtualización

- Process Level: permite lograr portabilidad entre diferentes sistemas. Java Virtual Machine (JVM)
- Storage Level: presenta una vista lógica del almacenamiento al usuario, quien no sabe dónde están los datos guardados (RAID, LVM, etc.)
- Network Level: integra recursos de hardware de red con recursos de software
- Operating System Level: SO permite la existencia de varias instancias de espacio de usuario aisladas (containers)
- System Level: permite la creación de máquinas virtuales



¿Por qué virtualizar?

- Tengo muchas máquinas servidores, poco usados
- Tengo que correr aplicaciones heredadas (legacy) que no pueden ejecutarse en nuevo hw o SO
- Tengo que probar aplicaciones no seguras
- Tengo que crear un SO o entorno de ejecución con recursos limitados
- Tengo que simular la computadora real, pero con un subconjunto de recursos
- Necesito usar un hw que no tengo (necesito “crear la ilusión” de hw)
- Necesito simular redes de computadoras independientes
- Tengo que correr varios y distintos SO simultáneamente
- Necesito hacer testeo y monitoreo de performance
- Necesito que SOs existentes se ejecuten en ambientes multi procesadores que comparten memoria
- Necesito facilidad de migración
- Necesito ahorrar energía (tendencias de green IT o tecnología verde)

Virtualización

Es una capa de abstracción sobre el hw para obtener una mejor utilización de los recursos y flexibilidad

Permite que haya múltiples máquinas virtuales (VM), o entornos virtuales (EV), con distintos (o iguales) sistemas operativos corriendo de manera aislada

Cada VM tiene su propio conjunto de hardware virtual (RAM, CPU, NIC, etc.) sobre el cual se ejecuta el SO “guest”

El SO “guest” ve un conjunto consistente de hw, no el hw real (aunque a través de ciertas configuraciones podría ver parte del hardware real)

VMs están representadas y son encapsuladas en archivos dentro de un File System

Fácil de almacenar, copiar

Fácil de hacer backup y restaurar

Simple de expandir y agregar recursos

Sistemas completos (aplicaciones ya configuradas, SO, hw virtual) pueden moverse de un servidor a otro rápidamente

Existe un software host (que simula) y un software guest (lo que se quiere simular)
Guest puede ser un sistema operativo completo



Monitor de Máquinas Virtuales (VMM)

Programa que se corre sobre el hardware para implementar las máquinas virtuales

Se encarga de controlar los recursos y de la planificación de los guests

Consideraciones:

- El VMM necesita ejecutarse en modo supervisor
- El software guest se ejecuta en modo usuario
- Las instrucciones privilegiadas en los guests implican traps al VMM
- El VMM interpreta/emula las instrucciones privilegiadas

Características de una VM (Popek y Goldberg)

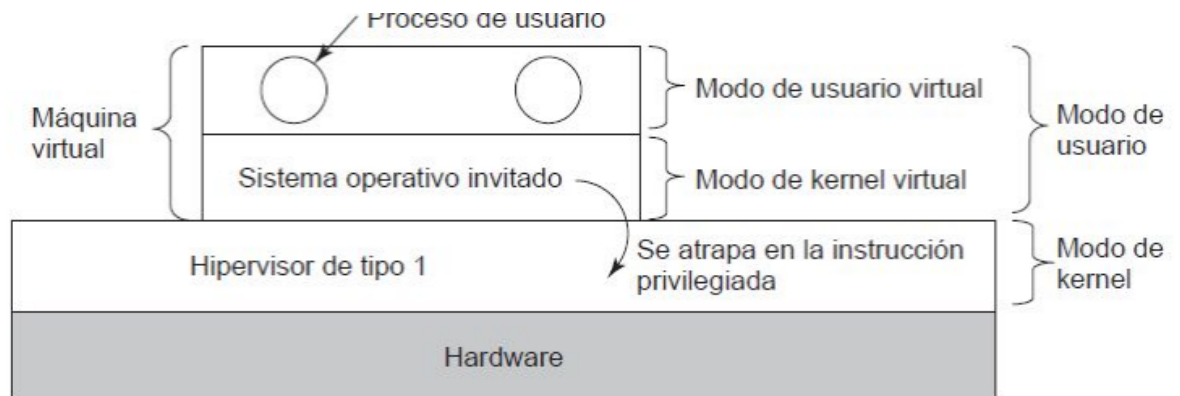
- Equivalencia/Fidelidad: un programa ejecutándose sobre un VMM (Virtual Machine Monitor), también conocido como Hipervisor, debería tener un comportamiento idéntico al que tendría ejecutándose directamente sobre el hardware subyacente
- Control de recursos/Seguridad: el VMM tiene que controlar completamente y en todo momento el conjunto de recursos virtualizados que proporciona a cada guest
- Eficiencia/Performance: una fracción estadísticamente dominante de instrucciones tienen que ser ejecutadas sin la intervención del VMM, o en otras palabras, directamente por el hardware

Instrucciones privilegiadas

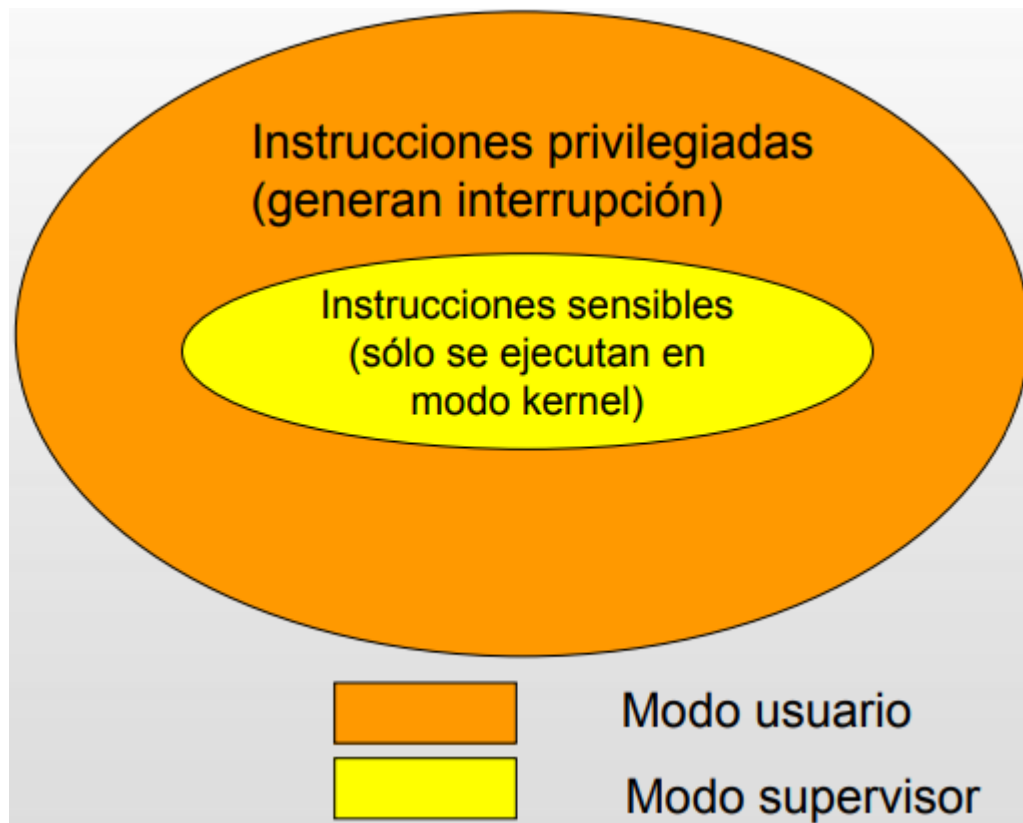
Cuando estoy virtualizando, el guest (corriendo en modo usuario) emitirá una instrucción privilegiada que NO debe ser ignorada, sino que debe generar un trap al SO

Mecanismo conocido como trap-and-emulate

No aplicable en arquitectura x86



- Instrucciones inocuas o no privilegiadas: se ejecutan nativamente
- Instrucciones privilegiadas: provocan una interrupción al ser ejecutadas en modo usuario
- Instrucciones sensibles: deben ejecutarse en modo kernel (E/S, configuración del hard (MMU), administración de interrupciones)



Para construir un VMM es suficiente con que todas las instrucciones que podrían afectar al correcto funcionamiento del VMM (instrucciones sensibles) siempre generen una excepción y pasen el control al VMM

Las instrucciones no privilegiadas deben ejecutarse nativamente en el hardware (es decir, eficientemente)

Para que un sistema soporte virtualización, las instrucciones sensibles deben ser un subconjunto de las privilegiadas (cosa que no pasaba en x86)

Anillos de privilegios en x86

Los procesadores x86 proveen protección basada en el concepto de niveles o anillos (rings) de privilegios (0 = más privilegio, 3 = menos privilegio)

Estos niveles se establecen en 2 bits del registro CS

El software más privilegiado se ejecuta en el Ring 0 (modo Kernel) y el menos privilegiado (modo usuario) en el 1, 2 o 3 (en particular en Intel se ejecuta en el 3)

El nivel de privilegios determina si las instrucciones privilegiadas, que controlan la funcionalidad básica de la CPU, pueden ejecutarse sin generar una excepción:

- En Ring 0 se ejecutan directamente
- En otros Rings se invocan a través de una excepción

Se busca que los hipervisores se ejecuten en el Ring 0 (o lo más cercano) y los SO guest se ejecuten en el 1 o 3 (x86)

Trap and emulate

Funciona similar a la emulación pero realiza una interpretación selectiva

Aplicaciones y sistema operativo (guest) ejecutan en modo usuario

Aplicaciones ejecutan nativamente en el hardware

VMM ejecuta en modo privilegiado

Cuando se ejecuta una instrucción privilegiada en el guest SO (en modo usuario) se produce un “trap” al VMM

VMM ejecuta las instrucciones necesarias y retorna el control al guest SO

No puede ser utilizado en todas las ISAs. Debe cumplir con el teorema de Popek and Goldberg

x86 no cumple con el teorema: no todas las instrucciones sensitivas son privilegiadas (por ej. popf)

Hipervisores

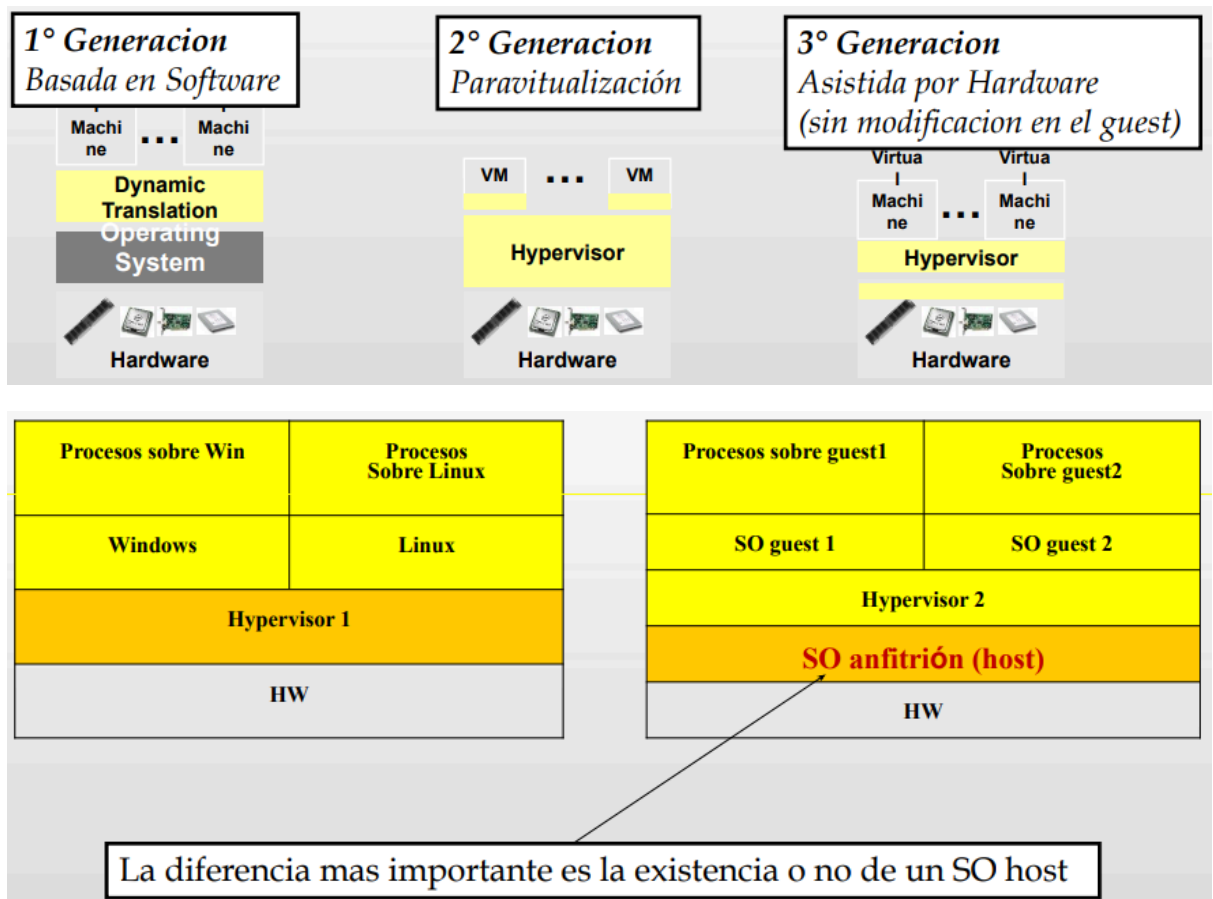
También conocidos como VMM (Virtual Machine Monitor) es una porción de software que separa las “aplicaciones/SO” del hardware subyacente

Proveen una plataforma de virtualización que permite múltiples SO corriendo en un host al mismo tiempo

Interactúan con el HW en líneas generales (2° y 3° generación)

Realizan la multiprogramación

Ofrece varias MV hacia arriba



Hipervisor tipo 1

Se ejecuta en modo kernel (Ring 0)

Cada VM se ejecuta como un proceso de usuario en modo usuario (Ring 3)

El SO guest no requiere ser modificado

Existen un modo kernel virtual y modo usuario virtual

Siempre que la VM ejecuta una instrucción sensible, se produce una trap que procesa el hipervisor. Algunos hipervisores introducen extensiones que le evitan tener que traducir todas las instrucciones

Debe tener asistencia de Hardware siempre

Hipervisor tipo 2

Se ejecuta como un programa de usuario en un SO host

Su función principal de interpretar un subconjunto de las instrucciones de hardware de la máquina sobre la que corre

Debe emularse el hardware que se mapea a los SO guest

Comparación

- Tipo 1:
 - Se ejecuta sobre el HW
 - Hipervisor se ejecuta en modo kernel real
 - SO guest en modo kernel “virtual” (pero es modo usuario)
- Tipo 2:
 - Se ejecuta como un programa de usuario sobre un SO host
 - Arriba de él, están los SO guests
 - Interpreta un conjunto de instrucciones de máquina
 - El SO host es quién se ejecuta sobre el HW

Técnicas de virtualización

De acuerdo al modo en el que se implemente el VMM podemos nombrar:

- Emulación
- Asistida por Hardware
- Paravirtualización

Emulación

Provee toda la funcionalidad del procesador deseado a través de software (ej: QEMU, MAME)

Se puede emular un procesador sobre otro tipo de procesador

Aplicación/SO emulado ejecuta en modo usuario

Se reescribe el conjunto completo de instrucciones

Todas las instrucciones son capturadas por el emulador

Cada instrucción es interpretada y traducida a una (o varias) equivalente, adecuada al hardware subyacente

Tiende a ser lenta

Full virtualization

Se trata de particionar un procesador físico en distintos contextos, donde cada uno de ellos corre sobre el mismo procesador

Los SO guest deben ejecutar la misma arquitectura de hardware sobre la que corren

No requiere que los guest se modifiquen

Es en general la técnica más utilizada

El VMM analiza el flujo de ejecución

Los bloques que contienen instrucciones sensibles son modificados

Los bloques con instrucciones inocuas se ejecutan directamente en el hardware

Traducción binaria

Introducida en 1998 por VMWare para permitir virtualizar sobre arquitecturas x86

Se combina traducción binaria con ejecución directa

Instrucciones no sensibles ejecutan directamente sobre el hardware

El hipervisor continuamente analiza en runtime el flujo de ejecución (bloques de código) de los SO guest y “traduce” las instrucciones sensibles por llamadas al hipervisor

Los bloques traducidos son ejecutados por la CPU directamente

Permite mejorar el rendimiento al poner en cache los bloques traducidos

Asistida por Hardware

Se necesitan CPU “virtualizables”

Intel la llama VT (Tecnología de virtualización) y AMD, SVM (máquina virtual segura)

Root-Mode y Non-Root Mode en Intel

Host Mode y Guest Mode en AMD

VMM ejecuta en root-mode, VMs en non-root mode

Ambos en ring-0

Cada modo 4 anillos de seguridad con diferentes niveles de privilegio

VMs generan traps al hipervisor cuando se ejecutan instrucciones sensibles (non-root mode)

Paravirtualización

Los hypervisors tipo 1 y 2 ejecutan SO guests no modificados

Se trata de tener SO guests modificados para mejorar el rendimiento

Cuando se quiere ejecutar una instrucción sensible, el SO guest la transforma en una llamada al VMM que expone una API específica

El VMM:

- No realiza traducción binaria completa
- No debe emular instrucciones de hardware, lo cual hace que las llamadas se resuelvan de modo más sencillo

El SO guest es como un proceso de usuario que hace llamadas al SO (el hipervisor)

El hipervisor cuenta con una API, que es un conjunto de llamadas a procedimientos

Los guests, en vez de invocar instrucciones sensibles, invocan a estas llamadas (hypercalls)

Se eliminan las instrucciones sensibles de los guest y se reemplazan por llamadas a la API especializada

El hipervisor se transforma en un microkernel

Decimos que un SO está paravirtualizado cuando se han eliminado, intencionalmente, algunas instrucciones sensibles (si se eliminan todas es paravirtualización completa si solo se eliminan algunas, se la llama paravirtualización parcial)

Si no se eliminan TODAS las instrucciones sensibles, el VMM deberá realizar traducción binaria

Implementaciones

Existen 2 modelos:

- Recompilando el Kernel del sistema guest:
 - Los drivers y la forma de invocar a la API residen en el kernel
 - Es necesario instalar un sistema operativo modificado/específico
- Instalando drivers paravirtualizados:
 - La paravirtualización es parcial (para algunas funciones y dispositivos)
 - Generalmente utilizada para placas de red, o gráficas
 - Esta es la técnica que se utiliza hoy en día

<i>virtualización “completa”</i>		<i>paravirtualización</i>	
Procesos usuario		Procesos usuario	
SO sin modificar (GNU/Linux, Windows)	Trap x instrucciones sensibles. Hypervisor emula y regresa.	Se accede a la API para todas o ciertas instrucciones. Se desacopla la función del hypervisor y VMM	SO modificado (GNU/Linux, Windows)
Hypervisor tipo 1		Microkernel	
HARDWARE			

Virtualización vs paravirtualización

Virtualización completa o nativa puede generar problemas de performance ya que tiene que emular la totalidad del hardware

Paravirtualización completa en kernel tiene mejor performance, pero soporta pocos SO, pues necesita modificar el SO original

Paravirtualización parcial en drivers es una solución intermedia

Clase 6 - cgroups, namespaces y containers

chroot

Chroot es una forma de aislar aplicaciones del resto del sistema

Introducido en la versión 7 de UNIX, 1979

Cambia el directorio a raíz aparente de un proceso. Afecta sólo a ese proceso y a sus procesos hijos

Al entorno virtual creado por chroot a partir de la nueva raíz del sistema se le conoce como "jail chroot"

No se puede acceder a archivos y comandos fuera de ese directorio

"chroot /new-root-dir comando"

chroot /usr/local/so ls /tmp

cgroups

En un sistema operativo se ejecutan varios procesos en forma concurrente

En Linux, por defecto, todos los procesos reciben el mismo trato en lo que respecta a tiempo de CPU, memoria RAM, "I/O bandwidth"

¿Qué sucede si se tiene un proceso importante que requiere prioridad?

O, ¿cómo limitar los recursos para un proceso o grupo de procesos?

El kernel no puede determinar cuál proceso es importante y cuál no

Existen algunas herramientas, como Nice, CPUlimit o ulimit, que permiten controlar el uso de los recursos por un proceso, pero, ¿son suficientes?

Control Groups, cgroups, son una característica del kernel de Linux que permite que los procesos sean organizados en grupos jerárquicos cuyo uso de varios tipos de recursos (CPU, memoria, I/O, etc.) pueda ser limitado y monitoreado

Desarrollo comenzado en Google por Paul Menage y Rohit Seth en el 2006 bajo el nombre de "process containers"

Renombrado en 2007 como "Control Groups"

Disponible desde la versión del kernel 2.6.24

Actualmente, Tejun Heo es el encargado del desarrollo y mantenimiento de CG

Versión 2 de CG con el Linux Kernel 4.5 de Marzo de 2016. Ambas versiones se habilitan por defecto

La interface de cgroups del kernel es provista mediante un pseudo-filesystem llamado cgroups

Permiten un control “fine-grained” en la asignación, priorización, denegación y monitoreo de los recursos del sistema

cgroups provee lo siguiente:

- Resource Limiting: grupos no pueden excederse en la utilización de un recurso (tiempo de CPU, cantidad de CPUs, cantidad de memoria, I/O, etc.)
- Prioritization: un grupo puede obtener prioridad en el uso de los recursos (tiempo de CPU, I/O, etc.)
- Accounting: permite medir el uso de determinados recursos por parte de un grupo (estadísticas, monitoreo, billing, etc.)
- Control: permite freezear y reiniciar un grupo de procesos

Procesos desconocen los límites aplicados por un “cgroup”

Actualmente hay 12 subsistemas definidos:

```
--- Control Group support
[*] Memory controller
[*] Swap controller
[ ] Swap controller enabled by default
[*] IO controller
-* CPU controller --->
[*] PIDs controller
[*] RDMA controller
[*] Freezer controller
[ ] HugeTLB controller
[*] Cpuset controller
[*] Include legacy /proc/<pid>/cpuset file
[*] Device controller
[*] Simple CPU accounting controller
[*] Perf controller
[*] Support for eBPF programs attached to cgroups
[ ] Debug controller
```

v1 vs v2

cgroups v1:

- Distintos controladores se han ido agregando en el tiempo para permitir la administración de distintos tipos de recursos
- En cgroups v1, desarrollo de los controladores fue muy descoordinado
- La administración de las distintas jerarquías se hizo cada vez más compleja

- Diseño posterior a la implementación

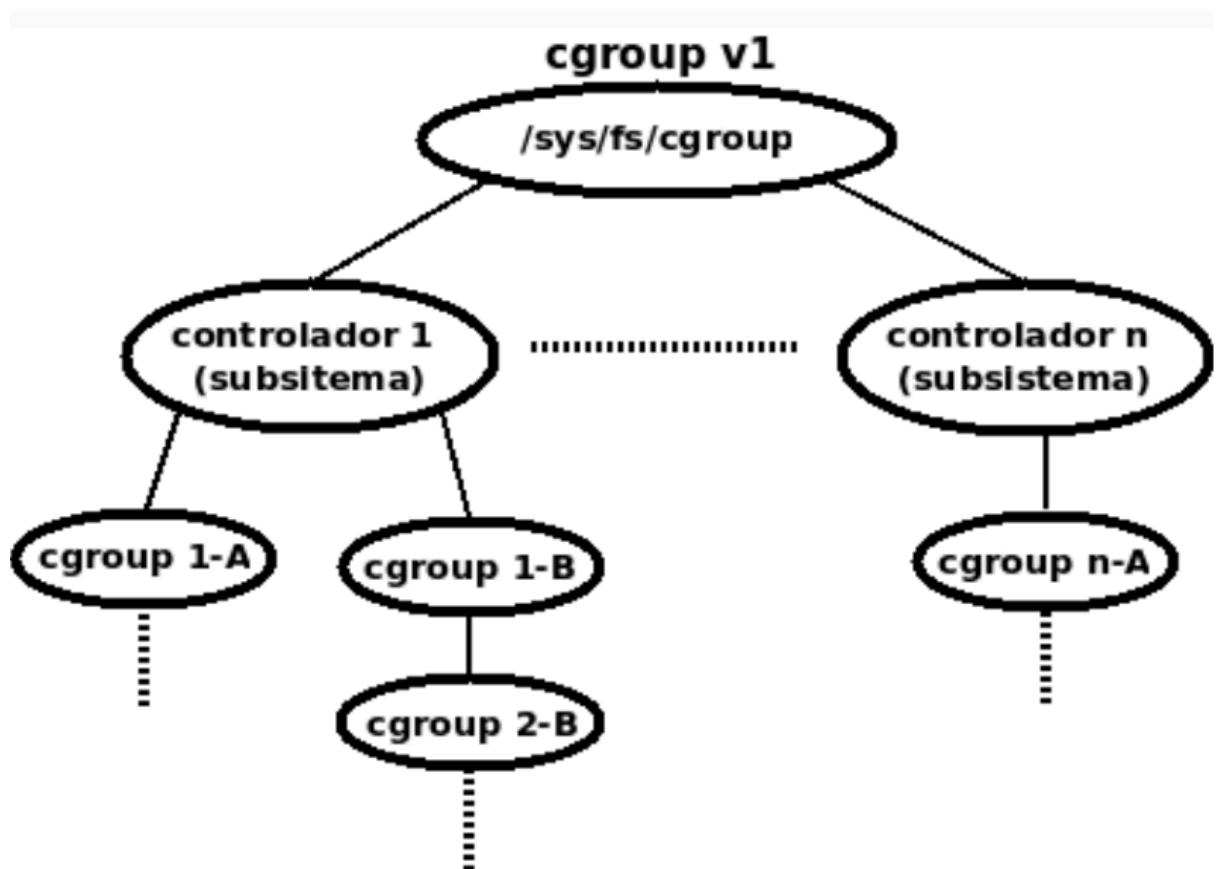
cgroups v2:

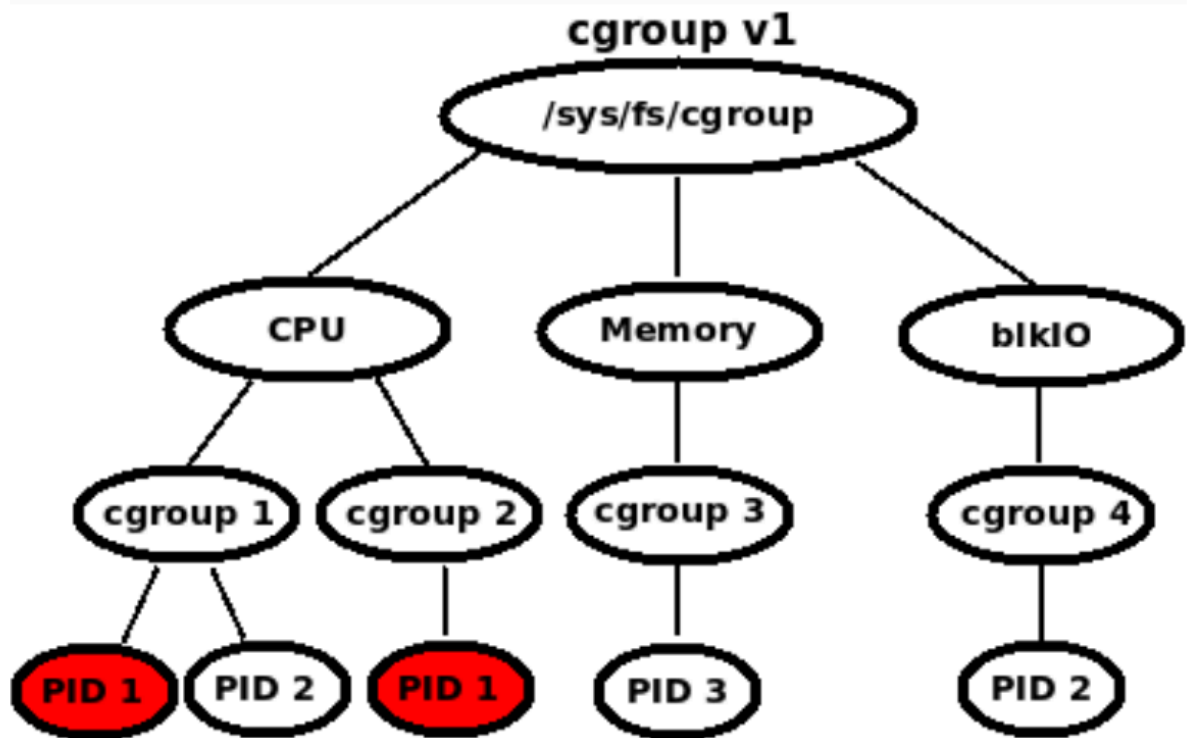
- Pensado como un reemplazo de cgroups v1
- Controladores también agregados en el tiempo
- Ambos controladores pueden ser montados en el mismo sistema
- Una jerarquía de un controlador no puede estar en ambos cgroups simultáneamente

cgroups v1

- cgroup: asocia un conjunto de procesos con un conjunto de parámetros o límites para uno o más subsistemas
- Subsistema: componente del kernel que modifica el comportamiento de los procesos en un cgroup. También llamado resource controllers o simplemente controllers
- Jerarquía: es un conjunto de cgroups organizados en una jerarquía
- Cada subsistema representa un único recurso: tiempo de CPU, memoria, I/O, etc.
- Cada jerarquía es definida mediante la creación, eliminación y renombrado de subdirectorios dentro del pseudo-filesystem
- Cada proceso del sistema solo puede pertenecer a un cgroup dentro de una jerarquía (pero a varias jerarquías)
- Los controladores pueden ser montados en pseudo-filesystems individuales o en un mismo pseudo-filesystem
- Por cada jerarquía, la estructura de directorios refleja la dependencia de los cgroups
- Cada cgroup es representado por un directorio en una relación padre-hijo. Por ejemplo: `cpu/procesos/proceso1`
- En cada nivel de la jerarquía se pueden definir atributos (por ej. límites)
- No pueden ser excedidos por los cgroups hijos
- Un proceso creado mediante un "fork" pertenece al mismo cgroup que el padre
- Cada directorio contiene archivos que pueden ser escritos/leídos
- Una vez definidos los grupos se le agregan los IDs de procesos
- Posible asignar threads de un proceso a diferentes cgroups. Deshabilitado en la v2, restaurado luego, pero con limitaciones
- Un controlador v1 debe ser montado contra un filesystem de tipo cgroup. Usualmente es mediante un filesystem tmpfs montado en `/sys/fs/cgroup`

- `mount -t cgroup -o cpu none /sys/fs/cgroup/cpu` para montar un controlador en particular (CPU en este caso)
- `mount -t cgroup -o all cgroup /sys/fs/cgroup` para montar todo los controladores
- Un controlador puede ser desmontado si no está ocupado: no tiene cgroups hijos (`umount /sys/fs/cgroup/cpu`)
- Cada cgroup filesystem contiene un único cgroup raíz al cual pertenecen todos los procesos
- `libcgroup`: tools para administrar los cgroups





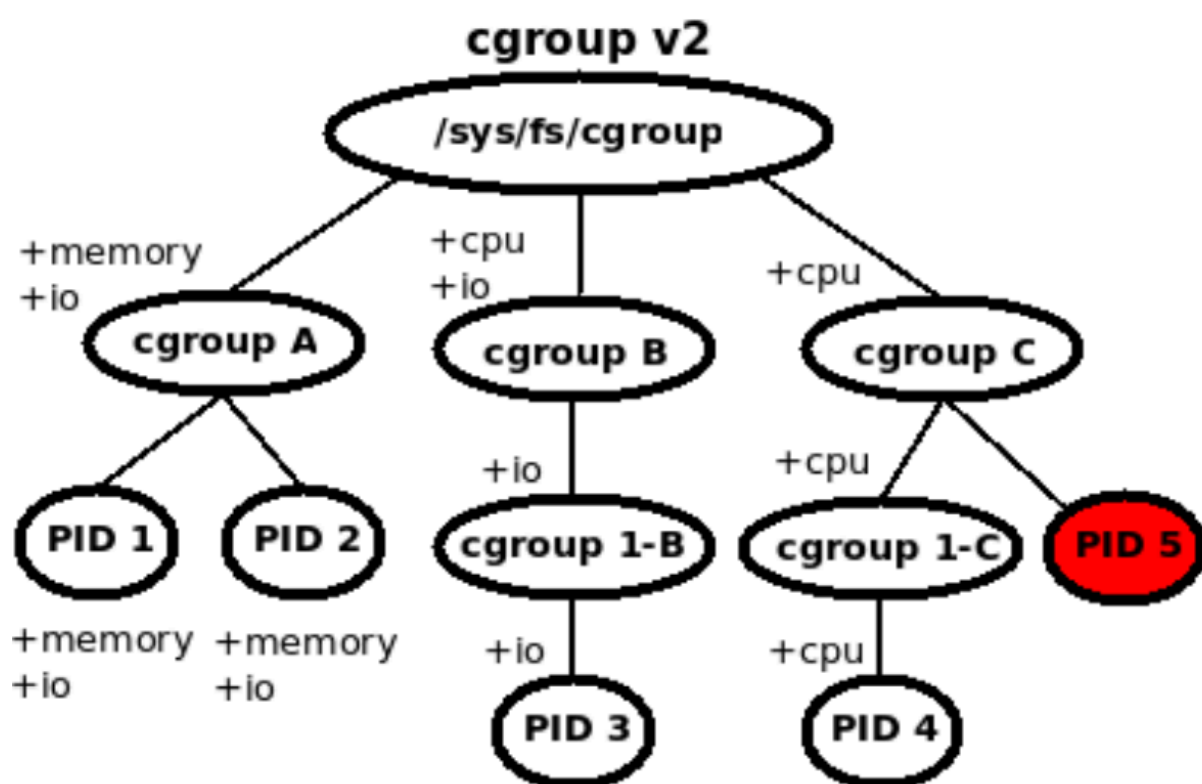
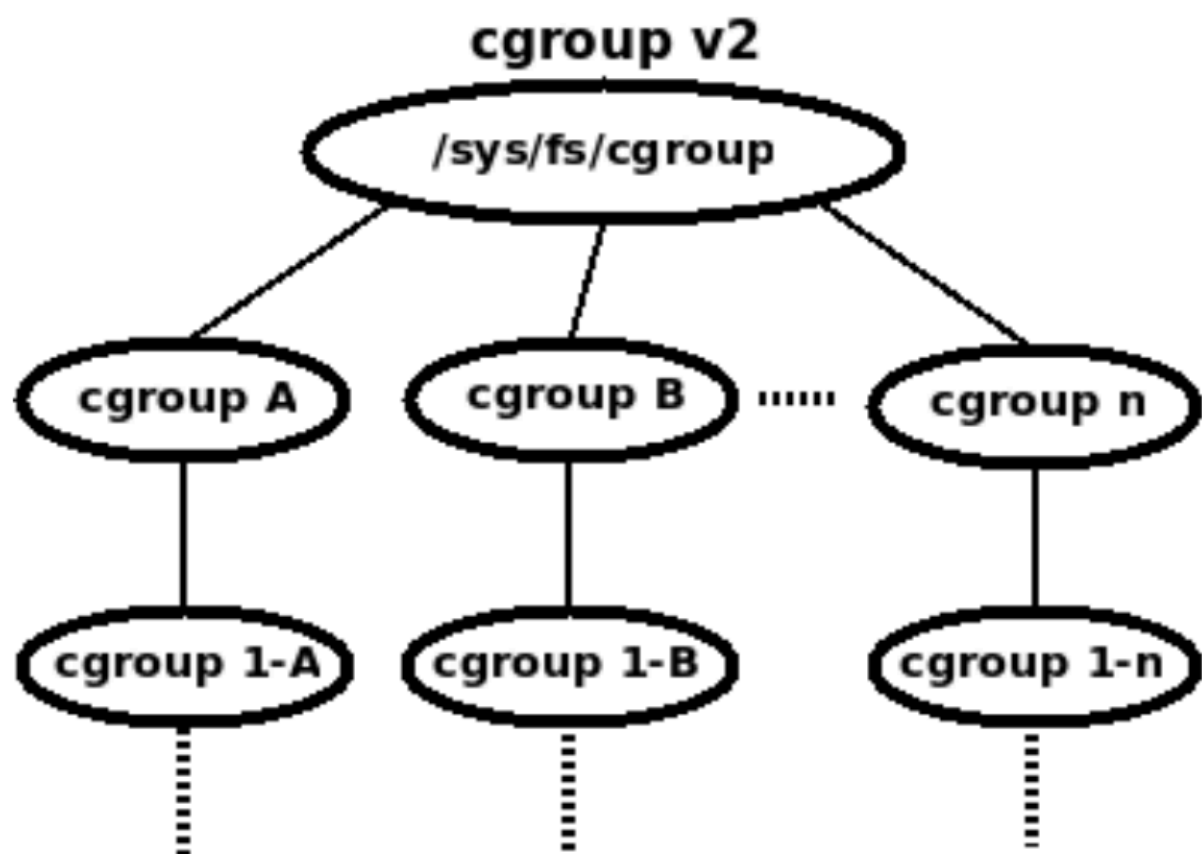
cgroups v2

- Todos los controladores son montados en una jerarquía unificada
- No es posible especificar un controlador en particular para montar
- Por ej.: `mount -t cgroup2 none /sys/fs/cgroup2`
- Todos los controladores son equivalentes a los de cgroups v1, excepto `net_cls` y `net_prio`
- Cada cgroup en la jerarquía v2 contiene, entre otros, los siguiente archivos:
 - `cgroup.controllers`: archivo de solo lectura que indica los controladores disponibles en un cgroup
 - `cgroup.subtree_control`: archivo de lectura/escritura que indica los controladores que se habilitarán en los cgroups hijos. Inicialmente vacío
- `cgroup.subtree_control` determina el conjunto de controladores que pueden ser usados en los cgroups hijos
- Los controladores que se pasan a los cgroups se indican en el archivo `cgroup.subtree_control`. + para habilitar, - para deshabilitar. Por ej.: `echo '+pids -memory' > cgroup.subtree_control`
- Un controlador que no está presente en el archivo `cgroup.controllers` no se puede agregar al archivo `cgroup.subtree_control`

- Los controladores disponibles en el archivo `cgroup.controllers` de un `cgroup` son idénticos a los existentes en el archivo `cgroup.subtree_control` del `cgroup` padre
- Si un controlador se remueve de un `cgroup` no puede ser rehabilitado en un `cgroup` hijo
- No se permiten procesos internos, a excepción del `cgroup` raíz:
 - Procesos deben asignarse a `cgroups` sin hijos (hojas)
 - Por ej. si `/procesos/proc1` son `cgroups` entonces un proceso puede residir en `/procesos/proc1`, pero no en `/procesos`
- Inicialmente, solo el `cgroup` raíz existe y todos los procesos pertenecen a él
- `mkdir CG NAME` y `rmdir CG NAME` para crear y eliminar `cgroups`
- Cada `cgroup` tiene un archivo lectura/escritura `cgroup.procs` con los procesos pertenecientes a ese `cgroup`. Inicialmente vacío
- Archivos correspondientes a un controlador habilitado en el archivo `cgroup.subtree_control` de un `cgroup` son automáticamente generados al crearse un `cgroup` hijo (controlador: `memory`, archivo: `memory_max`)
- En cada `cgroup` diferente al raíz existe un archivo `cgroup.events` de solo lectura
- Contiene pares de clave-valor:

```
$ cat proc1/cgroup.events
populated 1
frozen 0
```

- Populated: si es 1, este `cgroup` o alguno de sus descendientes tiene procesos miembros
- Frozen: si es 1, este `cgroup` está freezado
- Permite notificar cuando un `cgroup` está vacío



Namespaces

Permiten abstraer un recurso global del sistema para que los procesos dentro de ese “namespace” piensen que tienen su propia instancia aislada de ese recurso global

- Limitan lo que un proceso puede ver y, en consecuencia, lo que puede usar
- Modificaciones a un recurso quedan contenidas dentro del “namespace”
- Un proceso solo puede estar en un namespace de un tipo a la vez
- Un namespace es automáticamente eliminado cuando el último proceso en él finaliza o lo abandona
- Un proceso puede utilizar ninguno/algunos/todos de los namespace de su padre

Entre los namespaces provistos por Linux:

- IPC:
 - Flag: CLONE_NEWIPC
 - System V IPC, cola de mensaje POSIX
- Network:
 - Flag: CLONE_NEWNET
 - Dispositivos de red, pilas, puertos, etc.
- Mount:
 - Flag: CLONE_NEWNS
 - Puntos de montaje
- PID:
 - Flag: CLONE_NEWPID
 - IDs de procesos
- User:
 - Flag: CLONE_NEWUSER
 - IDs de usuarios y grupos
- UTS:
 - Flag: CLONE_NEWUTS
 - Hostname y nombre de dominio
- Cgroup:
 - Flag: CLONE_NEWCGROUP
 - cgroup root directory
- Time:
 - Flag: CLONE_NEWTIME
 - Distintos offsets al clock del sistema por namespace (Marzo 2020)

Flag: usado para indicar el namespace en las system calls

Tres nuevas systems-calls:

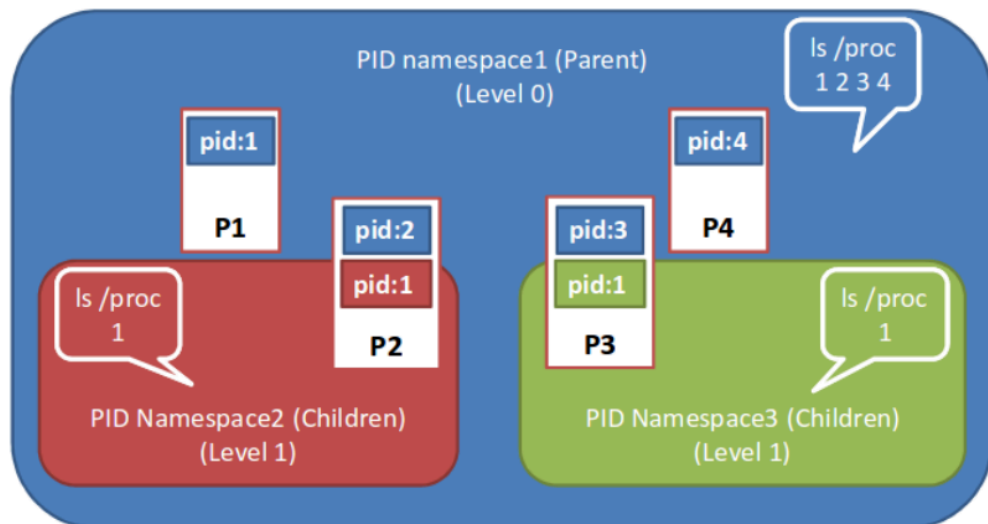
- clone():
 - Similar al fork
 - Crea un nuevo proceso y lo agrega al nuevo namespace especificado
 - Su funcionalidad puede ser controlada por flags pasados como argumentos
- unshare():
 - Agrega el actual proceso a un nuevo namespace
 - Es similar a clone, pero opera en el proceso llamante
 - Crea el nuevo namespace y hace miembro de él al proceso llamador
- setns():
 - Agrega el proceso actual a un namespace existente
 - Desasocia al proceso llamante de una instancia de un tipo de namespace y lo reasocia con otra instancia del mismo tipo de namespace

Cada proceso tiene un subdirectorío que contiene los namespaces a los que está asociado: /proc/[pid]/ns

Un proceso hijo hereda todos los namespaces de su proceso padre

PID namespace

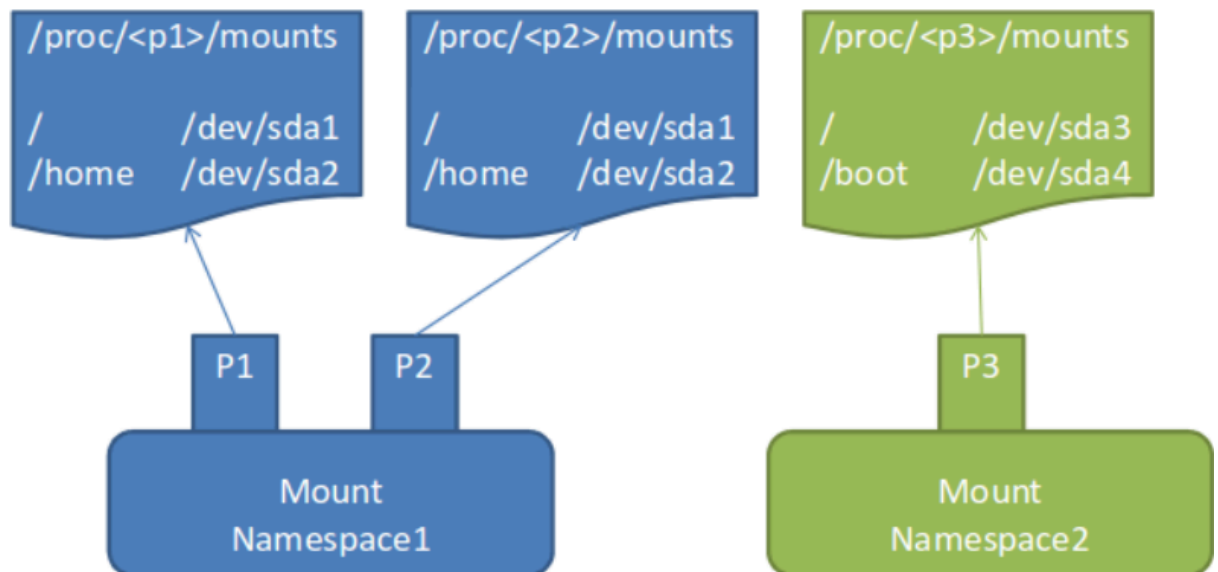
Brinda la posibilidad de tener múltiples árboles de procesos anidados y aislados



Mount namespace

Permite aislar la tabla de montajes (montajes por namespace)

Cada proceso, o conjunto de procesos, tiene una vista distinta de los puntos de montajes



User namespace

- Permite que los usuarios y grupos tengan sus propios IDs de usuario y grupo
- Usuarios y grupos también existen en los hosts, pero con diferentes IDs
- Usuario con ID 0 en el contenedor puede tener una identidad no root, no privilegiada, en el host

- No es necesario ser root para crear un user namespace
- Por default, usuario nobody en el contenedor
- /proc/PID/uid_map y /proc/PID/gid_map para indicar el mapping entre usuario y grupo en el host y en el contenedor
- ID-inside-ns ID-outside-ns length
 - ID-inside-ns: indica el UID inicial en el namespace
 - ID-outside-ns: indica el UID fuera del namespace
 - length: indica el número de mapeos de UID subsiguientes
 - Por ej: 0 1000 100

Containers

- Tecnología liviana de virtualización (lightweight virtualization) a nivel de sistema operativo que permite ejecutar múltiples sistemas aislados (conjuntos de procesos) en un único host
- Instancias ejecutan en el espacio del usuario. Comparten el mismo kernel (el del SO base)
- Dentro de cada instancia son como máquinas virtuales. Por fuera, son procesos normales del SO
- Método de virtualización más eficiente: mejor performance, booteo más rápido
- No es necesario un software de virtualización tipo hypervisor
- No es posible ejecutar instancias de SO con kernel diferente al SO base (por ej. Windows sobre Linux, aunque para eso se usa WSL)
- LXC, Solaris Zones, BSD Jails, Docker, Podman, etc.

Varias definiciones de container:

- Una forma de empaquetar aplicaciones
- Una máquina virtual liviana
- Un conjunto de procesos aislados del resto

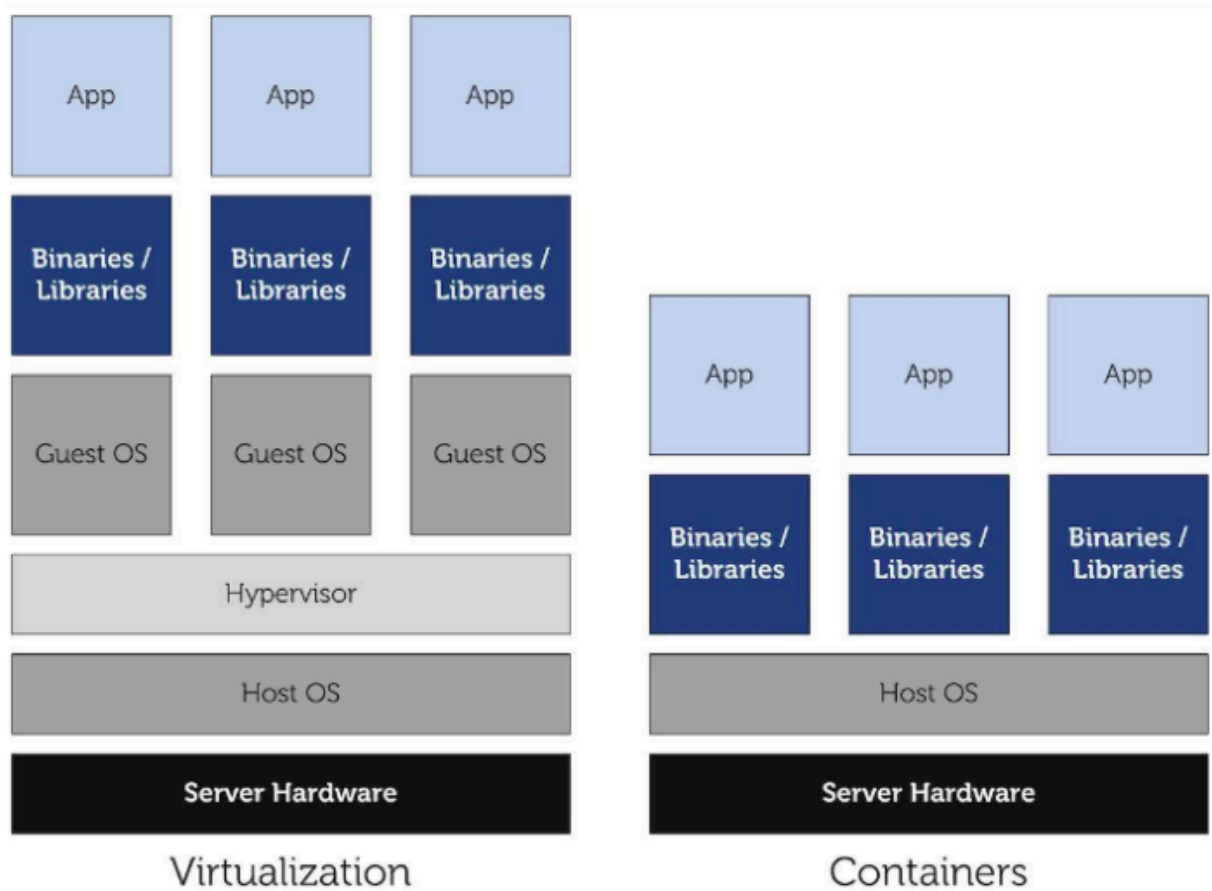
Dos tipos de contenedores:

- De sistemas operativos:
 - Ejecutan un SO completo (menos el kernel)
 - LXC, BSD Jails, Solaris Zone, etc.
- De aplicaciones:
 - Empaqueta una aplicación o proceso

- Docker, Podman, etc.

Sus principales características:

- Autocontenidos: tiene todo lo que necesita para funcionar
- Aislados: mínima influencia en el nodo y otros contenedores
- Independientes: administración de un contenedor no afecta al resto
- Portables: desacoplados del entorno en el que ejecutan. Pueden ejecutar de igual manera en diferentes entornos



- Containerization es la habilidad de construir y empaquetar aplicaciones como contenedores shippables
- Un contenedor contiene un código específico y todas las librerías y dependencias necesarias para ejecutarse
- Ejecutan de manera aislada en modo usuario usando un kernel compartido
- Procesos en un contenedor tienen 2 IDs: uno en el contenedor y otro en el host (PID Namespace)
- Típicamente, cada contenedor provee un único servicio ("micro-servicio")

- Desde el lado del nodo host, un contenedor es un proceso (o conjunto de procesos) ejecutándose
- En el nodo host se ven los procesos de todos los contenedores. Esto no es posible a la inversa ni entre distintos contenedores

Clase 7 - Docker

Docker es una plataforma open-source que permite empaquetar y ejecutar una aplicación en containers livianos

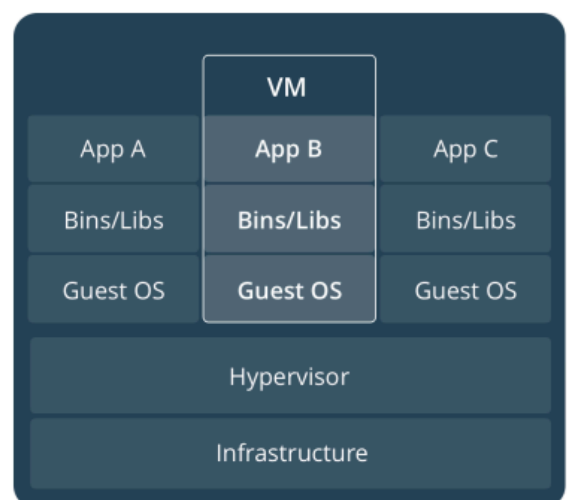
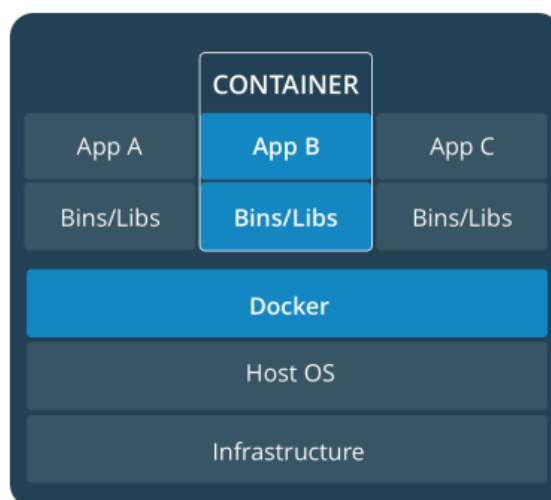
Docker Engine está dividido en 3 componentes:

- Docker Daemon (dockerd):
 - Es el servidor
 - Responsable de crear, ejecutar y monitorear los contenedores, construcción de imágenes, etc.
- API:
 - Especifica la interfaz que los programas pueden usar para interactuar con el servidor
- CLI:
 - Cliente
 - Permite a los usuarios interactuar con el servidor mediante comandos

Usos posibles:

- En desarrollo/testing
- Escalado y despliegue (deployment)
- Más servicios en un equipo sin VMs

Containers vs. VMs



¿Qué es Docker?

Utiliza una arquitectura cliente-servidor

Cliente y servidor pueden ejecutar en el mismo sistema (IPC o socket domain) o en diferentes nodos (socket TCP/IP)

Docker “daemon” escucha por “API requests”

Se comunican usando la REST API de Docker

Cliente envía comandos HTTP

Cliente y servidor se distribuyen como binarios

Ambos ejecutan en el espacio del usuario

Además de la CLI existe una interfaz gráfica: Docker Desktop

¿Cómo funciona?

Docker es una herramienta que utiliza una serie de características del kernel para proveer containers:

- Namespaces:
 - Docker lo utiliza para proveer el espacio de trabajo aislado que denominamos container
 - Por cada container Docker crea un conjunto de espacios de nombres (entre ellos pid, net, ipc y mnt)
- Control groups:
 - Para, opcionalmente, limitar los recursos asignados a un contenedor
- Union file systems:
 - Se utilizan como filesystem de los containers
 - Docker puede utilizar overlay2, AUFS, btrfs, vfs y DeviceMapper(deprecated)

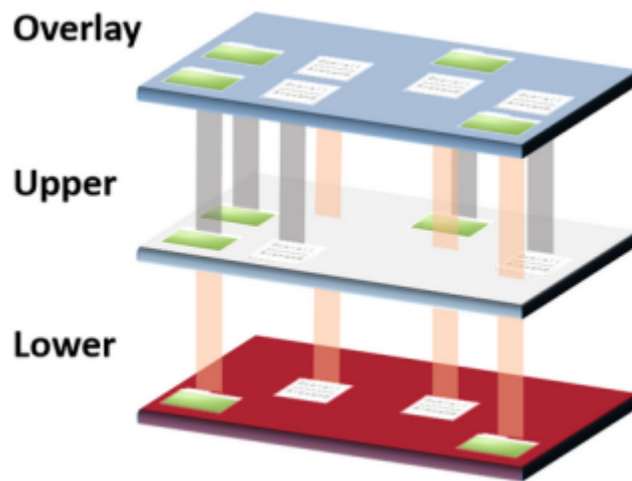
Union Mount Filesystems

Es un mecanismo de montajes, no un nuevo file system

Permite que varios directorios sean montados en el mismo punto de montaje, apareciendo como un único file system

Read-only capas inferiores, writable capa superior

Existen varias alternativas: UnionFS, AuFS, overlayFS, etc.



Definiciones

- imagen: template (molde) de sólo lectura con todas las instrucciones para construir un contenedor
- container: es una instancia de una imagen en ejecución
- registry: es un almacén de imágenes de Docker. Puede ser público o privado. Por defecto, docker utiliza Docker Hub
- Dockerfile: archivo de texto que indica los pasos necesarios para construir una imagen

Una imagen puede basarse en otras. Por ejemplo: `httpd` → `debian:jessie-backports` → `debian:jessie` → `scratch` donde `scratch` es un nombre especial que significa que se inicia desde una imagen vacía

Imágenes y capas

Cada imagen está compuesta de una serie de capas que se montan una sobre otra (stacking)

Cada capa es un conjunto de diferencias con la capa previa

Solo la última es R/W (la capa del container)

Las demás, de solo lectura (inmutables)

Las capas pueden ser reutilizadas entre las imágenes

La capa escribible permite almacenar datos generados durante la ejecución del contenedor

Docker usa “storage drivers” para almacenar capas de una imagen y para almacenar datos en la capa escribible de un contenedor

Distintos tipos de storage drivers: overlay2, btrfs, zfs, etc. En Windows, windowsfilter

Las capas pueden ser reutilizadas entre las distintas imágenes

Permiten extender las imágenes base de otros agregando nuevos datos

Apilando las capas:

- Cada capa que se baja, se extrae el contenido en un directorio del filesystem del nodo
- Al ejecutar el contenedor desde una imagen, se genera un union-filesystem donde las capas se apilan una sobre otra
- Usando chroot, se establece el union-filesystem creado como directorio raíz del contenedor
- Por último, se crea un nuevo directorio para el contenedor que permite modificar el filesystem (capa escribible de contenedor)
- Directorio modificable es el que permite correr múltiples contenedores a partir de las mismas capas (uno nuevo por cada contenedor)

DockerFile

Las imágenes se construyen siguiendo las instrucciones de un Dockerfile

```
FROM ubuntu:24.10
MAINTAINER user1 <user1@example.com>
RUN apt-get -y update && apt-get -y install nginx
COPY index.html /usr/share/nginx/html/
ENV APP_USER=nginx_user
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Cada capa en la imagen representa una instrucción (build instruction) (no todas, por ejemplo, MAINTAINER es solo informativa) en el archivo Dockerfile

Si un comando modifica el filesystem se genera una nueva capa

Al remover un archivo (RUN rm -f /home/user1/index.html) se genera una nueva capa, pero el archivo sigue existiendo en la capa anterior (sigue ocupando espacio)

Contenedores

Un contenedor es una instancia de una imagen

La principal diferencia entre un contenedor y una imagen es la capa escribible

Al eliminar un contenedor esa capa también se elimina

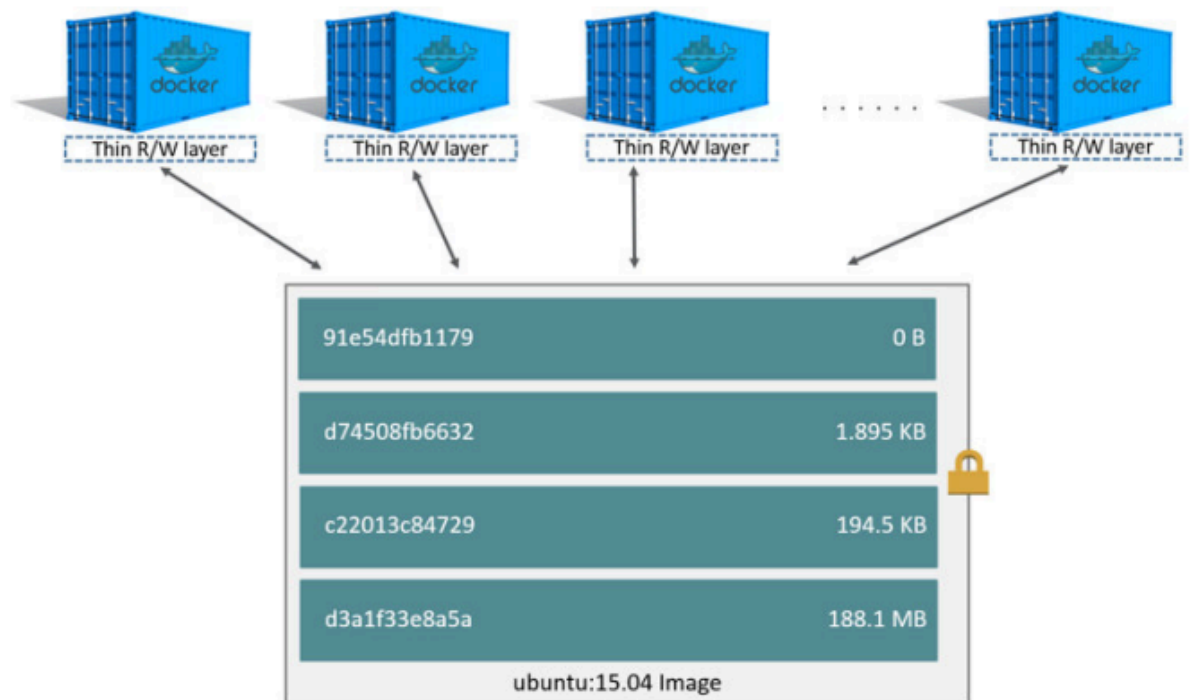
Las capas inferiores se mantienen sin modificaciones

Desde una imagen es posible generar varios contenedores

Cada contenedor es autónomo y ejecuta en su propio entorno aislado

Todos los contenedores comparten el mismo kernel y ejecutan en su propios namespaces

Los contenedores pueden ser iniciados, detenidos, pausados y destruidos usando la CLI de Docker



Almacenamiento (Storage)

Los archivos creados dentro de un contenedor son almacenados en una capa escribible (que se elimina al eliminar el container), por lo que los datos escritos en el contenedor no persisten cuando es destruido

Docker tiene dos opciones para almacenar datos en el host para que sean persistentes:

- Volumes: almacenados en una parte del filesystem administrada por Docker (por default: /var/lib/docker/volumes)
- Bind Mounts: pueden estar en cualquier parte del filesystem. Pueden ser modificados por procesos que no sean de Docker

Ambos deben ser montados en el contenedor

Los volúmenes permiten una mejor portabilidad entre sistemas, ya que para los bind mounts, el directorio del filesystem del host debe existir, cosa que podría no suceder siempre. En cambio los volúmenes los maneja docker, entonces no habrá problemas

Networking

Los contenedores tienen el networking habilitado por default (puede deshabilitarse)

Es posible realizar conexiones salientes

Los usuarios pueden definir nuevas redes

Múltiples contenedores pueden conectarse a la misma red y comunicarse usando direcciones IP y/o nombre

Un contenedor se puede conectar a varias redes simultáneamente

Para hacer disponible un servicio el contenedor debe publicar el correspondiente puerto

Dos contenedores en el mismo host no pueden publicar el mismo puerto

Los contenedores usan los mismos servidores DNS que el nodo host, pero se pueden modificar

Clase 8 - Docker compose y Podman

Docker compose

Docker Compose es una herramienta para correr aplicaciones que requieren múltiples contenedores

Contenedores son llamados servicios en Docker Compose

Facilita la creación de servicios, almacenamiento y red mediante un archivo YAML

El archivo YAML se ubica en el directorio de trabajo y recibe el nombre `compose.yaml` por default

También tiene comandos para iniciar, parar y construir servicios, monitorear los servicios en ejecución, logging, etc.

`docker compose up -d` para iniciar todos los servicios. `-d` para que ejecuten en background (versiones anteriores usaban el comando `docker-compose`)

Actualmente no es necesario indicar la versión (pero puede aparecer en antiguos archivos)

Cada servicio termina siendo un contenedor

Se puede indicar la política de reinicio del contenedor si se detiene por algún motivo

Permite definir la dependencia de arranque entre contenedores

Se puede indicar que el contenedor se debe crear a partir de la imagen creada desde un Dockerfile

Por default, compose establece una red default a la que todos los contenedores se unen

Es posible definir redes propias para cada compose

```

version: '3.9'
services:
  wordpress:
    container_name: wp-so
    image: wordpress
    ports:
      - 8080:80
    environment:
      - WORDPRESS_DB_NAME=wordpress
      - WORDPRESS_DB_USER=wordpress
      - WORDPRESS_DB_PASSWORD=wordpresspwd
    depends_on:
      - mysql
    networks:
      - backend
  mysql:
    image: mysql:8.0.39
    volumes:
      - /home/docker/mysql:/var/lib/mysql
    environment:
      - MYSQL_ROOT_PASSWORD=wordpressdocker
      - MYSQL_DATABASE=wordpress
      - MYSQL_USER=wordpress
      - MYSQL_PASSWORD=wordpresspwd
    restart: always
    networks:
      - backend
networks:
  backend:
    driver: bridge

```

Podman

Podman es una abreviación de Pod Manager

Es un container engine daemonless para desarrollar, administrar y ejecutar contenedores OCI (Open Container Initiative) en sistemas Linux

Un pod comprende uno o más contenedores que comparten los mismos namespaces

Utiliza prácticamente los mismos comandos que Docker (incluso tiene un podman compose)

Permite ejecutar imágenes con el formato OCI, tanto como Docker (v1 y v2)

Soporta todos los runtimes de OCI: runc, crun, etc.

También pueden ser ejecutados en Windows y MAC

No necesita un proceso demonio central para administrar los contenedores

Los contenedores inician como procesos estándar del sistema

Basado en la librería libpod que contiene toda la lógica necesaria para instrumentar el ciclo de vida de un contenedor:

- Formato de las imágenes, tanto Docker como OCI
- Autenticación, descarga y almacenamiento de imágenes desde una registry, construcción de nuevas imágenes, etc.
- Ciclo de vida de los contenedores: crear, ejecutar, eliminar, etc. contenedores
- Manejo tanto de simple contenedores como de pods
- Aislamiento de los contenedores/pods (mediante cgroups a bajo nivel)
- CLI para administración de los contenedores/pods
- Soporte de contenedores/pods rootless
- libpod interactúa con los runtimes

Pod

Concepto que proviene de Kubernetes en el que los contenedores se ejecutan en Pods

Uno o más contenedores trabajando en conjunto con un propósito en común

Representa un conjunto de contenedores que comparten almacenamiento y una única IP

Servicios en los contenedores dentro del pod se pueden comunicar entre sí usando localhost

Pods se pueden crear vacíos y luego agregarle contenedores

Ventajas de agrupar dos o más contenedores:

- Compartir algunos namespaces y cgroups
- Compartir el volumen para almacenar datos persistentes
- Compartir la misma configuración
- Compartir el mismo IPC

Cada pod incluye un contenedor llamado infra

También se lo suele llamar el pause container

Su finalidad es mantener abiertos los namespaces asociados con el pod

Al agregarse un contenedor al pod, los procesos comparten varios namespaces del pod

Al compartir el net namespace, los procesos se comunican usando localhost (127.0.0.1)

Es posible iniciar/detener un pod y/o un contenedor dentro de un pod

La mayoría de los atributos se asignan al contenedor infra: port binding, namespaces, cgroups

Si se desea cambiar un atributo se debe regenerar el pod (por ej. agregar un contenedor que escuche en un nuevo puerto)

Por cada contenedor dentro del pod existe un proceso common

Conmon es un programa C liviano que monitorea un contenedor hasta que finaliza

Es una herramienta de comunicación entre el container engine (Podman) y el OCI runtime (runc o crun)

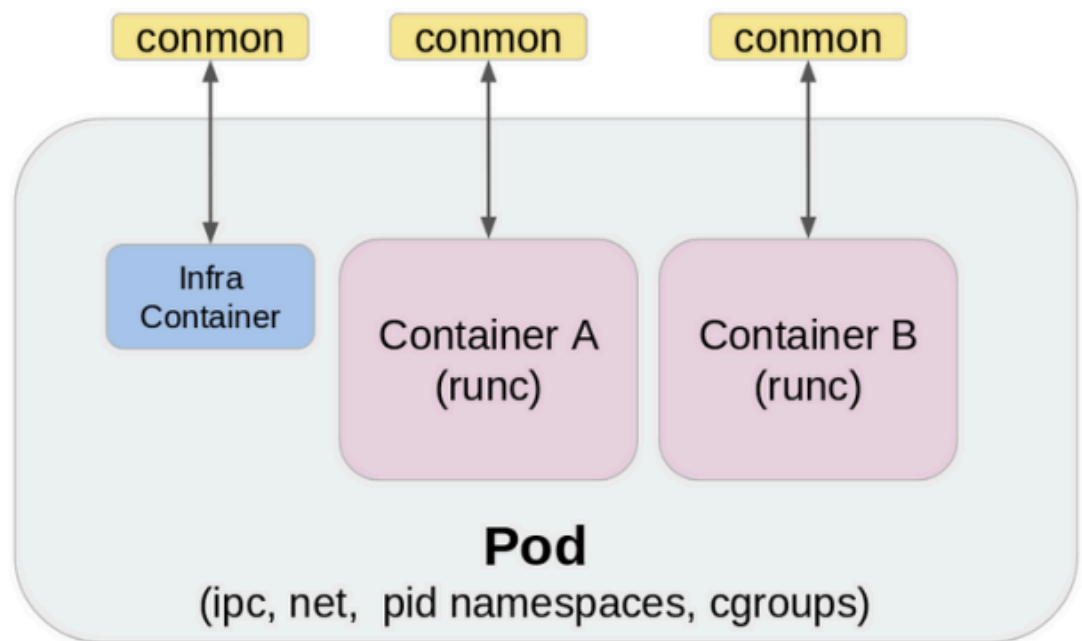
Ejecuta el runtime, indicándole donde se encuentra el archivo OCI spec y el rootfs (capa que será el punto de montaje en el contenedor)

Su principal tarea es monitorear el proceso principal del contenedor

Salva el código de salida si el contenedor muere

Mantiene la tty del contenedor abierta para poder conectarse a él

Los contenedores comparten los namespaces de tipo PID, networking e IPC:



Estandarización

Containerd es el runtime usado por Docker Engine (en conjunto con runc)

En 2015, Docker, en conjunto con RedHat, AWS, Google, etc., inicia el Open Container Initiative (OCI) auspiciado por la Linux Foundation

OCI se encarga de realizar la especificación de runtime y de imágenes

También lanzó la primera implementación de un runtime de contenedores que cumplen con esa especificación: runc

Además, la OCI definió la base para una más directa conexión en Kubernetes y el correspondiente engine

La comunidad de Kubernetes libera CRI (Container Runtime Interface), plugin que permite la adopción de una amplia variedad de runtimes

En 2017, RedHat libera CRI-O que permite el uso de runtimes compatibles con OCI

Es una alternativa liviana a usar Docker, rkt, etc.

Orquestador

Cluster: grupo de nodos interconectados que trabajan en conjunto

Permite aprovisionar, desplegar, escalar y administrar automáticamente contenedores sin preocuparse por la infraestructura subyacente

Creación de servicios de manera declarativa

En general, dos tipos de nodos:

- Manager: encargado de administrar el cluster
- Worker: encargado de ejecutar las aplicaciones

Service Discovery: orquestador brinda información para encontrar otro servicio

Routing: paquetes deben llegar entre servicios ejecutando en diferentes nodos

Load Balancing: distribuir las cargas de trabajo entre las distintas instancias de un servicio

Scaling: aumentar/disminuir las instancias de un servicio según la carga de trabajo

Ejemplos: Kubernetes, Docker Swarm, RedHat Openshift, Rancher, etc.

Clase 9 - Seguridad

Los recursos informáticos deben protegerse frente a accesos no autorizados, destrucciones maliciosas o introducción accidental de incoherencias

Los recursos a proteger son (entre otros):

- Datos/Información
- CPU
- Memoria
- Dispositivos

El responsable de llevar a cabo la tarea es, entre otros, el Sistema Operativo, a través de un conjunto de mecanismos

En base a la correcta o incorrecta aplicación de los mecanismos de protección, podremos determinar el nivel de seguridad con el que cuenta el sistema

Protección ≠ seguridad

Protección: mecanismos específicos del SO para resguardar la información dentro de una computadora, para controlar el acceso de los procesos (o usuarios) a los recursos existentes

Seguridad: medida de la confianza en que se puede preservar la integridad de un sistema y sus datos. Es un concepto más general

Seguridad

La seguridad utiliza distintos mecanismos con el fin de proteger y garantizar ante:

- Amenazas:
 - Confidencialidad de los datos (Intercepción / Modificación)
 - Integridad de los Datos (Modificación)
 - Disponibilidad (Interrupción)
- Intrusos:
 - Acceso indebido al sistema o datos
- Pérdida accidental de datos:
 - Accidentes Naturales
 - Errores de HW o SW

- Errores humanos

Protección

Acceso al sistema (1) vs. Acceso a recursos en un sistema (2):

- (1) se resuelve a través de:
 - Autenticación
 - Control sobre una base de datos de usuarios
- (2) se resuelve a través de:
 - Permisos
 - Control de acceso (obligatorio o no)

Políticas ≠ mecanismos

Es importante definir las políticas antes que los mecanismos

Las políticas:

- Son el qué
- Definen lo que se quiere hacer, en base a los objetivos
- Las podemos asociar a los papeles
- Rara vez incluyen configuraciones

Los mecanismos:

- Son el cómo
- Definen cómo se hace
- En este punto aparecen las configuraciones e implementaciones reales
- Hay diferentes mecanismos para cumplir una política

Esta separación garantiza la flexibilidad de un sistema

Objetos y dominios

Un sistema informático es una colección de procesos y objetos

Objetos: de HW (CPU, Memoria, etc.) o de SW (archivos, programas, semáforos)

Cada objeto debe tener un identificador único que permita referenciarlo

Los procesos pueden realizar un conjunto finito de operaciones sobre los objetos

Un dominio es un conjunto de pares (objeto, derecho)

Cada par especifica un objeto y un subconjunto de operaciones que se pueden realizar con él

Un derecho (right) significa autorización para efectuar esas operaciones

Por ejemplo:

- El dominio D contiene la pareja (archivo A, {read,write})
- Un proceso que se ejecuta dentro del dominio D puede leer y grabar el archivo A

¿Quiénes pueden ejecutarse en un dominio?:

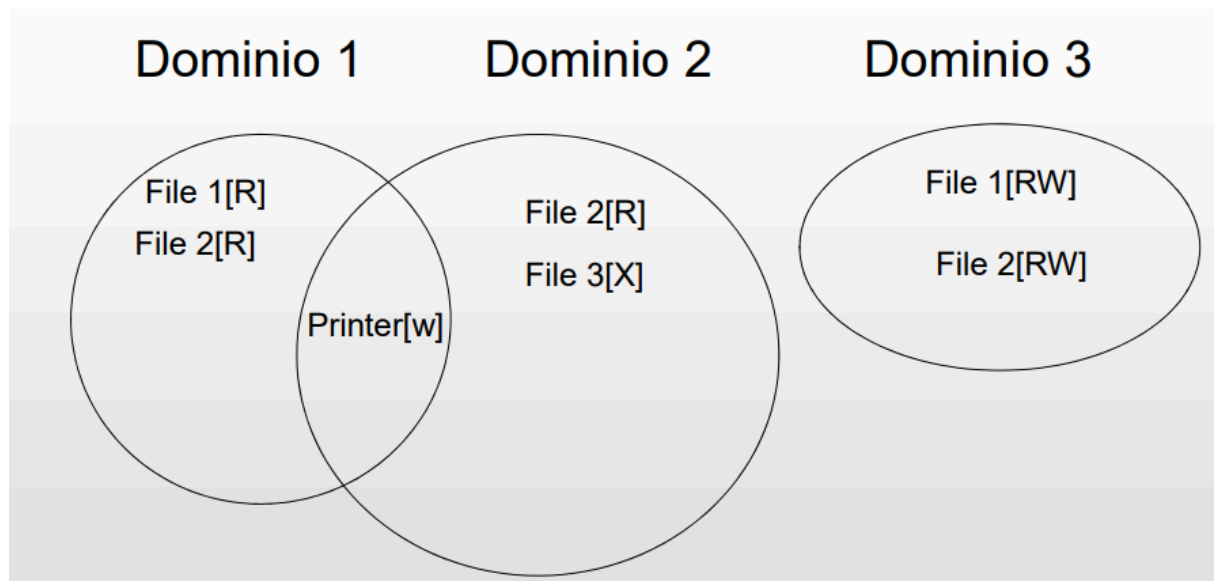
- Puede ser un usuario y define qué puede hacer ese usuario (por defecto lo que no se permite se deniega)
- Puede ser un proceso y el conjunto de objetos a los que podrá acceder dependerá de la identidad del proceso
- Puede ser un procedimiento y definirá el conjunto de variables a las que puede acceder (variables locales, globales, etc...)

Dominios y procesos

Principio de need-to-know o POLA (Principle of least authority): define que los procesos accedan sólo a los objetos que necesitan (con los derechos que necesiten) para completar su tarea

La relación entre un proceso y un dominio puede ser estática o dinámica:

- Relación estática:
 - Si el conjunto de objetos a los que el proceso accede durante su ciclo de vida es fijo
 - Siempre mismo dominio
 - Puede generar que los procesos tengan más privilegios que los que necesitan en sus fases de ejecución
- Relación dinámica:
 - Si el conjunto de objetos puede variar
 - Puede cambiar de dominio
 - Por ejemplo usando los bits SETUID y SETGID en UNIX sobre los archivos



Un conjunto de programadores pueden pertenecer al mismo dominio

En Unix el dominio lo definen el UID y el GID

Dado (UID, GID) hay un conjunto de objetos a los que se puede acceder con determinados permisos

Dos procesos con igual (UID, GID) pueden acceder al mismo conjunto de archivos: mismo dominio

Matriz de acceso

Controla la pertenencia de objetos a dominios y sus derechos

Las filas representan dominios

Las columnas representan objetos

Cada elemento $\text{access}(i, j)$ representa el conjunto de operaciones (derechos) que un proceso puede invocar en un objeto O_j dentro del dominio D_i

Implementa las políticas de protección/seguridad de un sistema

Objeto \ Dominio	File1	File2	File3	Printer
D1	Read	Read		Print
D2		Read	execute	print
D3	Read/write	Read/write		

Operación switch

Un proceso puede cambiar de un dominio a otro

Para poder cambiar de un dominio a otro se debe habilitar la operación switch sobre un objeto (que va a ser un dominio en este caso)

La matriz en sí es un objeto que posee atributos

De esta manera se define si sobre un dominio se realiza asignación estática o no: la conmutación del dominio D_i al dominio D_j estará permitida si se encuentra definida en el switch access(i, j)

Dominio y matriz son objetos sobre los cuales se definen accesos

Objeto \ Dominio	File1	File2	File3	Printer	D1	D2	D3
D1	Read	Read		Print		switch	
D2		Read	execute	print			
D3	Read/write	Read/write			switch		switch

Operación copy

Es un derecho que se asocia a un elemento $\text{access}(i, j)$ de la matriz

Indica que un proceso ejecutándose en ese dominio puede copiar los derechos de acceso de ese objeto dentro de su columna

Se denota con *

Objeto Dominio	File1	File2	File3	Printer
D1	Read	Read		Print
D2		Read *	execute	print
D3	Read/write			
Objeto Dominio	File1	File2	File3	Printer
D1	Read	Read		Print
D2		Read *	execute	print
D3	Read/write	Read		

Variantes

- Transferencia: si un derecho se copia desde $\text{matriz}(i, j)$ a $\text{matriz}(k, j)$, el derecho desaparece para $\text{matriz}(i, j)$, o sea, el derecho fue transferido
- Propagación o copia limitada: se copia el derecho pero no el derecho a copia en el nuevo (R^* es copiado como R , no como R^*) (en el ejemplo se ve así)

Operación owner

Permite agregar nuevos derechos y borrar los ya existentes

Si $\text{matriz}(i, j)$ incluye el derecho de owner entonces un proceso ejecutándose en el dominio D_i puede agregar y borrar cualquier entrada en la columna j

Objeto \ Dominio	File1	File2	File3	Printer
D1	Read	Read		Print
D2		Read * owner	execute	print
D3	Read/write			
Objeto \ Dominio	File1	File2	File3	Printer
D1	Read	Read/ write		Print
D2		Read * owner	execute	print
D3	Read/write	Read		

Operación control

Indica que pueden modificarse y borrarse derechos dentro de una fila

La operación control es aplicable sólo a dominios

Si matriz (i, j) incluye el derecho de control, entonces un proceso ejecutándose en el dominio Di puede remover cualquier derecho de acceso dentro de la fila j

Objeto \ Dominio	File1	File2	File3	Printer	D1	D2	D3
D1	Read	Read		Print		switch	control
D2		Read	execute	print			
D3	Read/ Write	Read/ write			switch		switch

Cualquier proceso que se ejecute en el dominio D1, puede controlar los Switch Access de cualquier columna del dominio D3

Resumen

Copy, owner y control son operaciones que se utilizan para controlar cambios al contenido de la matriz de acceso

Switch y Control: son aplicables sólo a dominios

Copy y owner: puede modificar derechos dentro de una columna

Control: puede modificar derechos dentro de una fila

Implementación

La forma de representar y almacenar la matriz de acceso, generalmente no es bajo el formato de matriz, ya que se desperdiciaría mucho espacio (es ineficiente)

El principal problema es que puede tener muchos elementos vacíos

Generalmente se almacenan sólo los elementos ocupados utilizando 2 métodos:

- Almacenarla por filas
- Almacenarla por columnas

Se debe optimizar el acceso para que sea rápido

Alternativas de implementación:

- Tabla Global: la más simple
- Lista de Control de Acceso por objetos (por columna)
- Lista de Capacidades por Dominio (por fila)

Tabla global

Es la más sencilla de implementar

Consiste en conjunto de tuplas

Cada vez que se ejecuta una operación M sobre un objeto Oj sobre el dominio Di, se analiza la tabla y se verifica si se encuentra una terna $\langle D_i, O_j, M \rangle$

Si se encuentra se permite la operación

Si no se encuentra se deniega

Su principal desventaja es que el tamaño de la tabla hace que no se pueda almacenar toda en memoria

Lista de control de acceso (ACL)

Consiste en asociar con cada objeto una lista (ordenada) que contenga todos los dominios que pueden acceder al objeto, y la forma de hacerlo

Cada columna de la matriz se puede ver como una lista de acceso a un objeto, descartando elementos vacíos

Para cada objeto, hay una lista de pares ordenados

Cuando se intenta realizar una operación M sobre un objeto O_j en el dominio $D_{i(A, B, C)}$, se busca en la lista en el objeto O_j una entrada $\langle D_i, R_k \rangle$, donde M pertenece al conjunto R_k



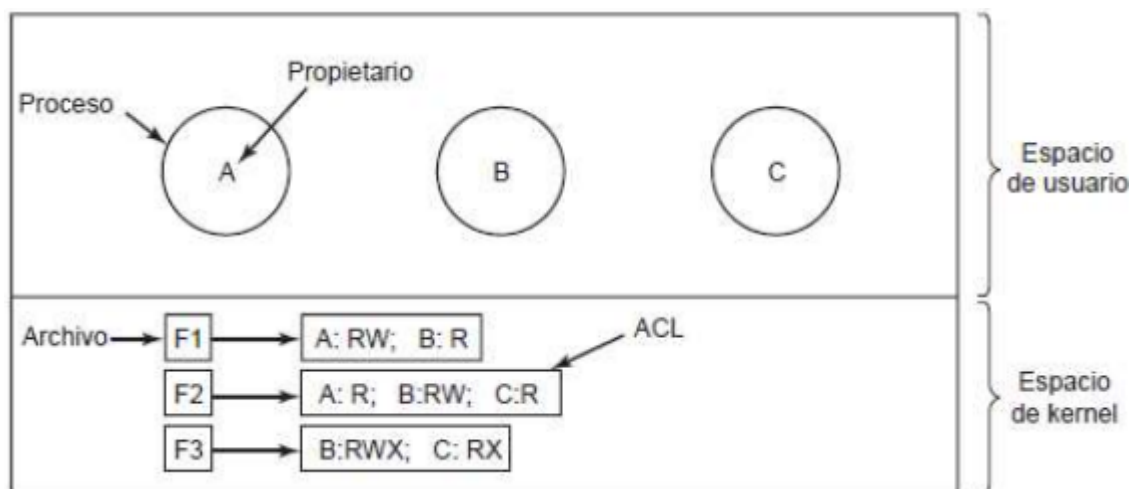
Cada archivo tiene una ACL asociada

El archivo F1 tiene dos entradas en su ACL

La primera indica que cualquier proceso que sea propiedad del usuario A puede leer y escribir en el archivo

La segunda indica que cualquier proceso que sea propiedad del usuario B puede leer el archivo

Todos los demás accesos están prohibidos



Lista de capacidades

Es una lista de objetos del dominio con sus derechos (división x filas)

A esta lista se le conoce como lista de capacidades y a los elementos individuales que contiene se les conoce como capacidades

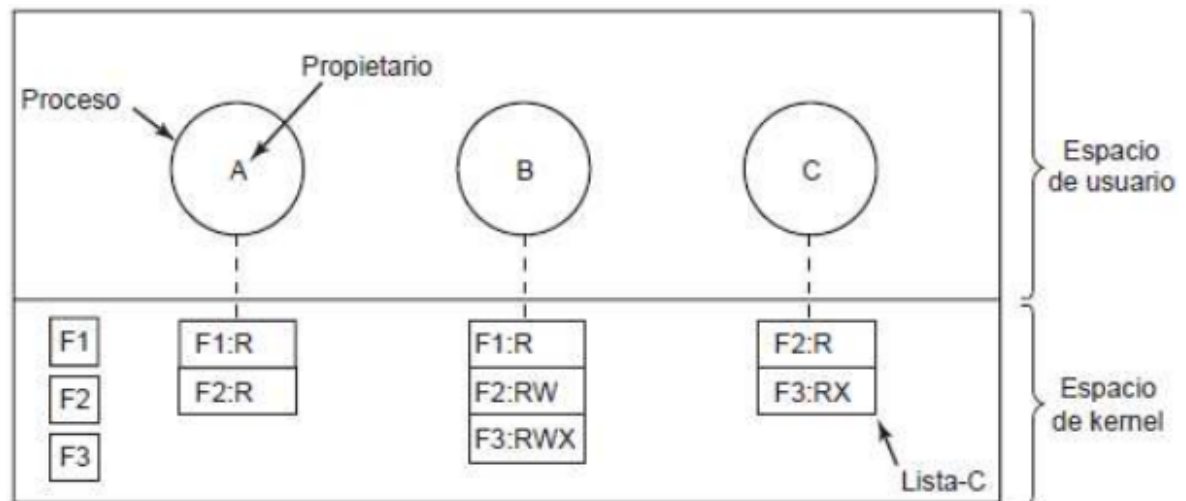
El proceso no la accede directamente

Esta lista es un objeto protegido, a la que sólo accede el SO

Cada proceso tiene una lista con los objetos que puede utilizar, junto con qué operaciones (el dominio)

Presenta dificultades al momento de revocar o modificar un permiso sobre un objeto: se deben recorrer todas las listas de capacidades

Cada capacidad otorga al propietario ciertos derechos sobre un objeto



Por ejemplo, el proceso que pertenece al usuario A puede leer los archivos F1 y F2

Sistemas confiables

Otro factor que afecta el nivel de seguridad de un sistema, es el código:

- Código malintencionado: virus, gusanos, troyanos, etc.
- Código con errores de programación: backdoors

¿Es posible construir sistemas seguros? La respuesta es sí, pero el gran enemigo de la seguridad es el agregado de funcionalidad. Un sistema minimalista será probablemente muy seguro, pero pobremente usable. Un sistema con muchas

características, probablemente tenga más errores y abra más la posibilidad a que sea vulnerado

Explotación de errores en código

Los procesos, junto con el Kernel son una potencial amenaza a la seguridad de un sistema

Los atacantes aprovechan errores en la codificación del SO, o algún proceso con alto nivel de privilegios con el fin de que los mismos cambien su funcionamiento normal:

- Desbordamiento de buffer
- Cadenas de formato
- Retorno a libc
- Desbordamiento de enteros
- Inyección de código

Clase 10 - Multiprocesadores

Origen

Desde su inicio, la industria de las computadoras se ha orientado fundamentalmente a buscar un poder de cómputo cada vez mayor

Las necesidades actuales demandan cada vez mayor poder de cómputo (física, astronomía, biología, cálculo de modelos, IA)

En el pasado, la solución era siempre hacer que el reloj operara a mayor velocidad

En la actualidad, lograr mayor velocidad es físicamente complejo:

- Ninguna señal eléctrica se puede propagar más rápido que la velocidad de la luz
- Problemas de disipación de calor (muchos transistores juntos en poco espacio)
- Problemas de consumo eléctrico

La solución al problema es el cómputo en paralelo y/o distribuido

Contar con varias CPU que operen a velocidad “normal” y que en conjunto provean la potencia de cómputo necesaria

Esquemas

Si tenemos que resolver un problema en una única CPU, el esquema de trabajo es sencillo

Si tenemos varios problemas y varias CPU, a priori podríamos asignar estáticamente una tarea a cada una de ellas, pero no es lo más eficiente

Deberá existir un coordinador que se encargue de repartir las tareas

Al existir múltiples CPU, la complejidad aumenta en lo que refiere a distribución de tareas, pasaje de mensajes y acceso a memoria

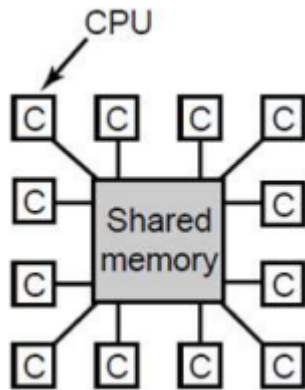
Multiprocesadores con memoria compartida

La comunicación entre las CPU es a través de la memoria compartida

Cada CPU tiene el mismo acceso que otras a la memoria física a través de un único BUS físico

Para acceder a una palabra de memoria por lo general cada CPU requiere de 2 a 10 nanosegundos

Existe un único espacio lógico de direcciones para todos los procesos

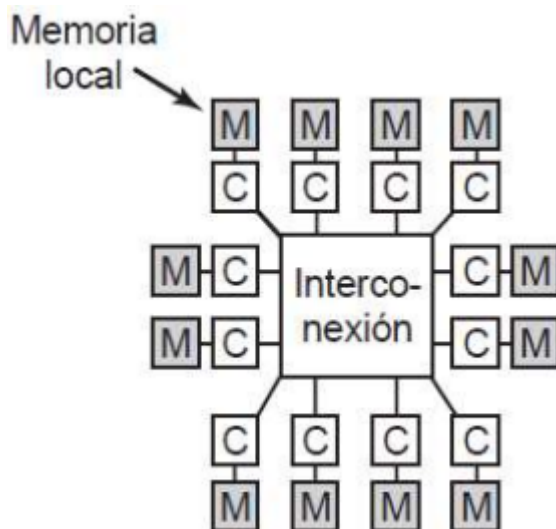


Multicomputadora con memoria independiente/pasaje de mensajes

Varios pares de CPU-memoria se conectan a una interconexión de alta velocidad pasando mensajes

Cada memoria es local para una sola CPU y puede ser utilizada sólo por esa CPU

El retardo del paso de mensajes es de entre 10 a 50 microsegundos



Son más sencillas de fabricar, pero más complejas de programar

También se los conoce como fuertemente acoplado, ya que una tarea se inicia y deseablemente debe terminar de ejecutarse en la misma CPU

Sistemas distribuidos

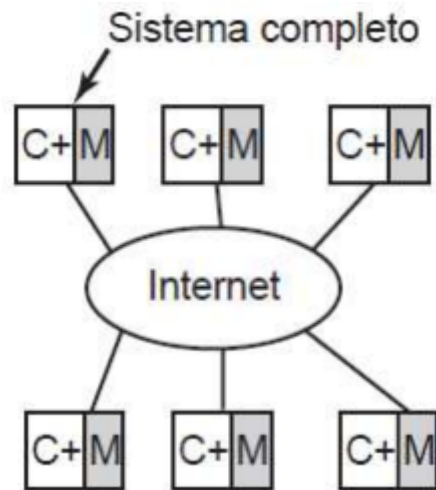
Conecta sistemas de cómputo completos a través de una red

Cada sistema tiene su propia memoria, y se comunican mediante el paso de mensajes

El retardo del paso de mensajes es de entre 10 a 100 milisegundos

Cada nodo de cómputo es una computadora completa

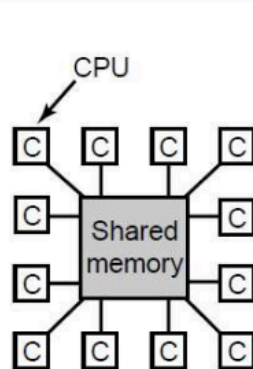
Heterogeneidad de sistemas y hardware



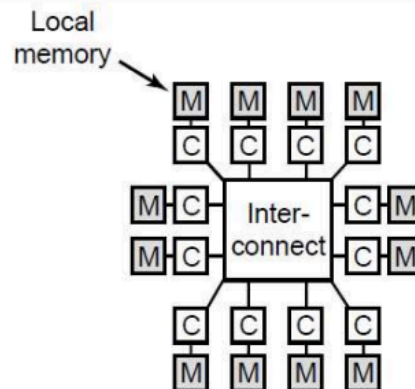
2 a 10 nseg

10000 a 50000 nseg

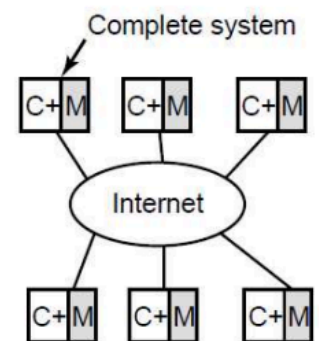
10 a 100 mseg



(a)



(b)



(c)

Chips multinúcleo

A medida que la tecnología avanza, los transistores se hacen más pequeños y aparece la posibilidad de agregar más de uno a un chip

Al tener más transistores se puede:

- Agregar más memoria caché, aunque está demostrado que la tasa de aciertos no se incrementa demasiado
- Agregar más velocidad de clock a una CPU. sigue existiendo un único hilo de ejecución
- Agregar más CPU al mismo chip (núcleos), los que podrían compartir la caché y la memoria principal. Se logra paralelismo

Es importante que el software se diseñe teniendo en cuenta los aspectos del hardware para aprovecharlo al máximo

Tipos de SO multiprocesador

Hasta el momento hicimos un análisis del hardware multiprocesador

Analicemos como el hardware es manejado por los distintos Sistemas Operativos (software)

Existen diversas metodologías posibles para la administración en esquemas multiprocesadores desde el lado del software

Responsabilidades del Sistemas Operativo

Por lo general, los sistemas operativos realizan tareas regulares como lo son:

- Manejo de System Calls
- Administración de memoria
- Administración de E/S (sist. de archivos y dispositivos)

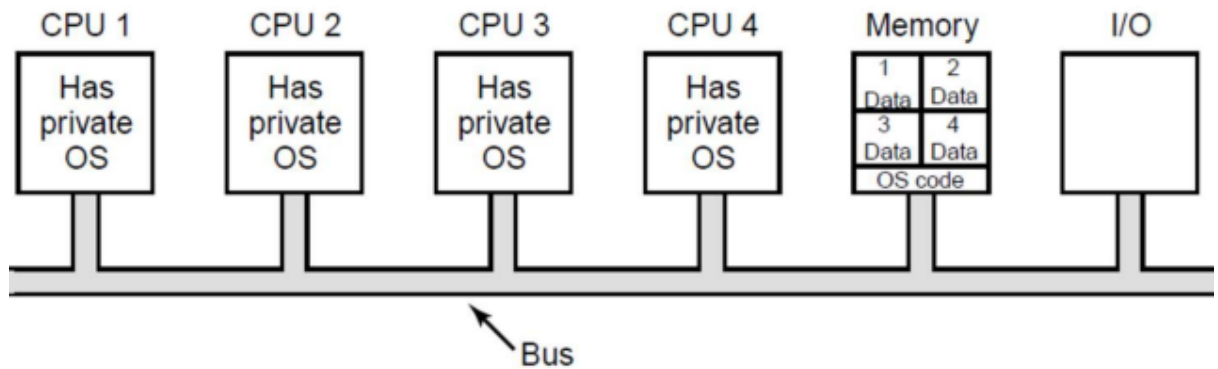
Algunas características particulares en sistemas multiprocesadores:

- Sincronización de procesos
- Administración de Recursos
- Planificación de CPU

Todas estas características se ven afectadas en sistemas multiprocesadores

Hay que atacar las problemáticas que pueden ocurrir

Cada CPU con su SO (modelo poco utilizado)



Se divide estáticamente la memoria para cada CPU con su copia privada (las CPUs operan independientes)

Se comparte el código de SO

Cada CPU cuenta con su propio conjunto de procesos

Desbalance en la carga de trabajo

Hay planificación independiente por CPU

Cada CPU atrapa y maneja las SysCalls de sus procesos

Los procesos quedan "atados" a una única CPU

No se pueden compartir páginas

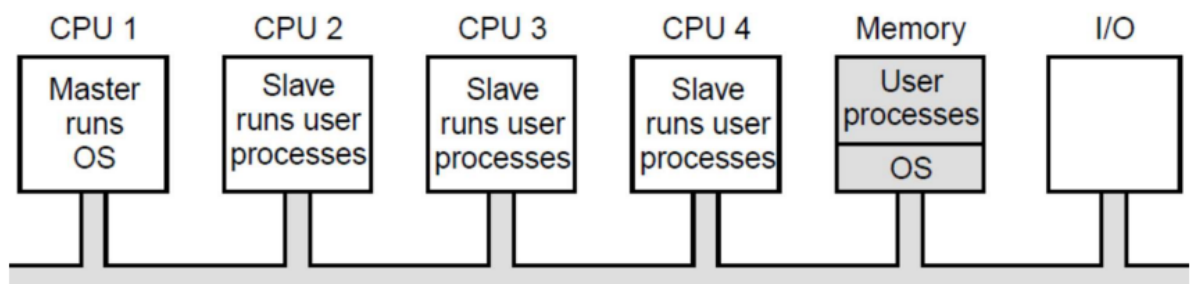
Se debe usar pasaje de mensajes

Memoria desperdiciada

Caché de disco, cada CPU tiene su propia copia ⇒ inconsistencia de la información

Se deben usar múltiples colas de procesos listos (una por CPU)

Maestro - Esclavo



Única copia del SO y de su información

Todas las SysCalls se redirigen a una CPU

Esta CPU puede ejecutar procesos si "le sobra tiempo"

Resuelve problemas del modelo anterior

Se mantiene una única cola de planificación

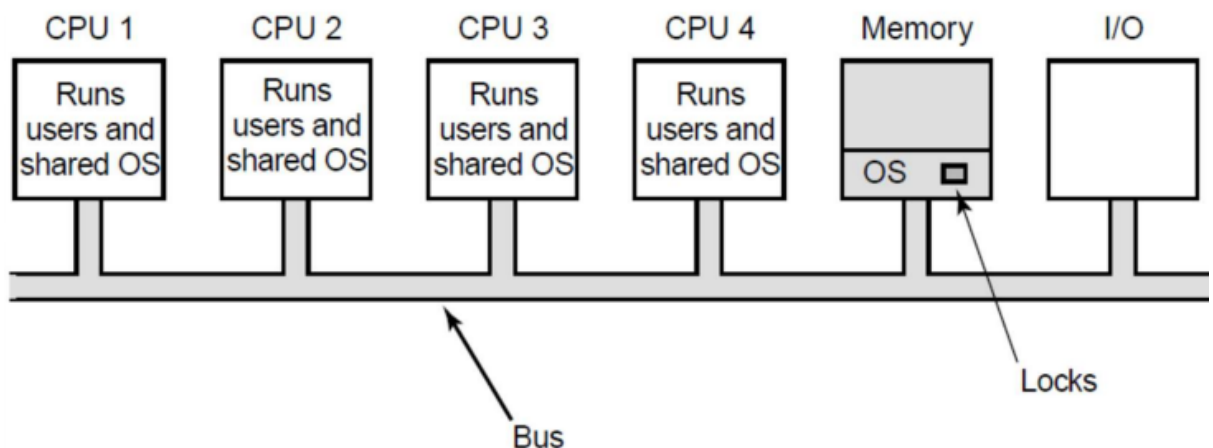
Cuando una CPU está libre, pide un proceso al master

Se pueden asignar las páginas entre todos los procesos de manera dinámica

Problema, con muchas CPUs hay un cuello de botella en el Maestro

Ej: Si el 10% del tiempo se atienden SysCalls, con 10 CPU el master se saturaría, con 11 estaría sobrecargado

SMP - Multiprocesadores Simétricos



Soluciona el inconveniente de saturación de una única CPU

Única copia del SO en memoria y cualquier CPU puede ejecutarlo

Cuando se invoca una System Call, es ejecutada por la CPU que la invocó

Provee equilibrio entre procesos y memoria, ya que solo hay un único conjunto de tablas del SO

No hay cuello de botella, ya que no hay una CPU master

Problemas:

- Dos o más CPUs ejecutando código del SO en un mismo instante de tiempo
- Dos CPUs podrían seleccionar el mismo proceso para ejecutar, o seleccionar la misma página de la memoria libre!

Posibles soluciones a los problemas planteados:

1. Utilizar “locks” para las estructuras del SO:
 - Considerar a todo el SO como una gran sección crítica
 - Cualquier CPU puede ejecutar el SO, pero solo una a la vez
 - Se comportaría como el modelo maestro esclavo
 - Es un modelo poco utilizado, debido a la mala performance que provee
2. Lock por estructura(s):
 - Existen varias secciones críticas independientes, cada una protegida por su propio mutex
 - Mejora el rendimiento
 - Dificultad para determinar cada sección crítica
 - Ciertas estructuras pueden pertenecer a más de una sección crítica, lo cual ante bloqueos podría generar Deadlocks
 - Es el esquema que generalmente se utiliza

Sincronización de multiprocesadores

Es necesario que las CPU de un multiprocesador se encuentren sincronizadas (acceso a regiones críticas, estructuras, etc.)

En entornos uniprocador si un proceso realiza una llamada al sistema que requiera acceder a cierta tabla crítica del kernel, el código del kernel sólo tiene que deshabilitar las interrupciones antes de tocar la tabla

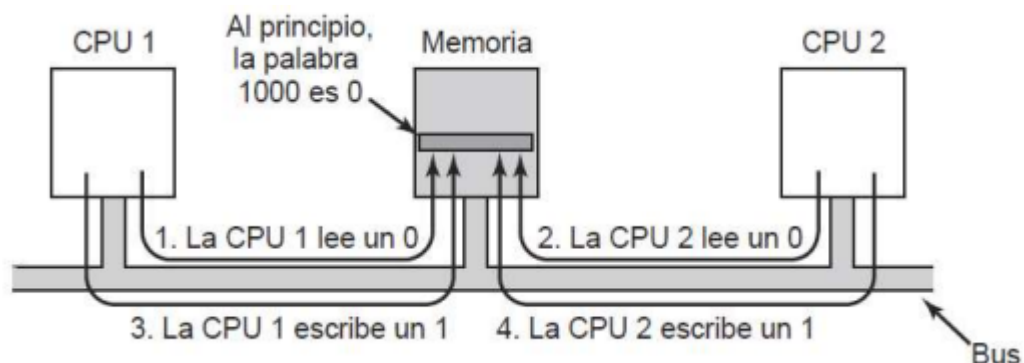
En sistemas multiprocesadores, se deshabilitan las interrupciones de una CPU, pero otra podría generarlas

Surge la necesidad de contar con un protocolo de mutex apropiado para garantizar la exclusión mutua

Una posibilidad para garantizar la exclusión mutua es el uso de TSL (Probar y establecer bloqueo (test and set lock)):

- Lee la palabra de memoria y la almacena en un registro
- Al mismo tiempo escribe un 1 en la memoria para hacer el lock (2 accesos al BUS)
- Cuando termina libera (escribe 0)
- En uniprocadores esta implementación es correcta
- El problema surge en entornos multiprocesadores: (la operación no es indivisible)

- Ambas CPU obtuvieron un 0 de la instrucción TSL, por lo que ambas tienen acceso a la sección crítica



La solución al problema anterior, es que en multiprocesadores la instrucción TSL bloquee el acceso al BUS

Se necesita soporte de hardware para poder implementarlo

Genera Carga en la memoria y el BUS, ya que la CPU que solicita debe mantener el bloqueo y las otras CPU deben esperar liberación del bloqueo

No es lo más eficiente...

Existen otras soluciones más eficientes, pero se debe tener soporte del Hardware

Planificación de multiprocesadores

Antes de analizar cómo se va a hacer la planificación, es necesario determinar que se va a planificar:

- Uniprocador:
 - ¿Qué hilo (o proceso) se seleccionará?
- Multiprocador:
 - Se suma ¿En que CPU se va a ejecutar?
 - Complejidad extra con hilos que trabajan "en conjunto":
 - Si son ULT, el planificador los planifica a nivel de proceso
 - Si son KLT, es posible tomar decisiones sobre su planificación

Planificación de hilos KLT independientes

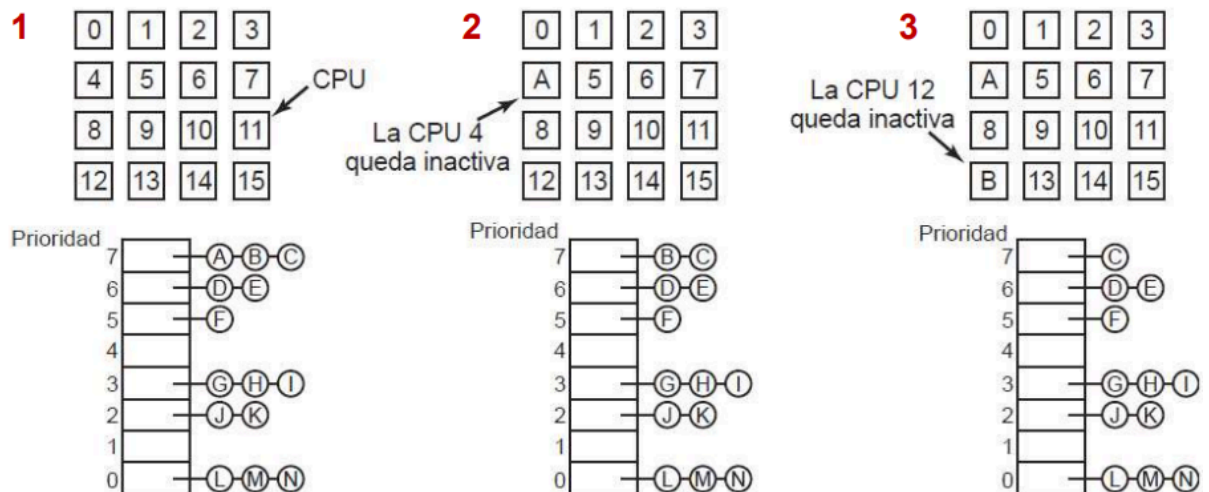
Esta situación se da generalmente en ambientes de tiempo compartido

Muchos usuarios ejecutando tareas que generalmente no tienen relación entre sí

El esquema más sencillo para planificarlos es tener una única cola de listos para todos los hilos

Podríamos tener varias: una para cada prioridad

Las 16 CPU están ocupadas y hay 14 hilos en la cola de listos ordenados por prioridad:



Aprovechamiento de la caché

Un hilo que se ejecuta en un CPU donde ya se ha ejecutado, tendrá mayor posibilidad de que sus datos aún sigan en la caché de dicha CPU → planificación por afinidad (Vaswani y Zahorjan, 1991)

Se utiliza el algoritmo de Planificación de 2 niveles:

- Cuando se crea un hilo, se asigna a una CPU
- Cada CPU tiene su propia colección de hilos y los planifica por separado
- Si queda una CPU ociosa, se reparten los hilos
- Minimiza la contención de las estructuras de datos asociadas para la planificación, ya que no hay solo una (una o más cola de listos por cpu)

Planificación de hilos que trabajan en conjunto

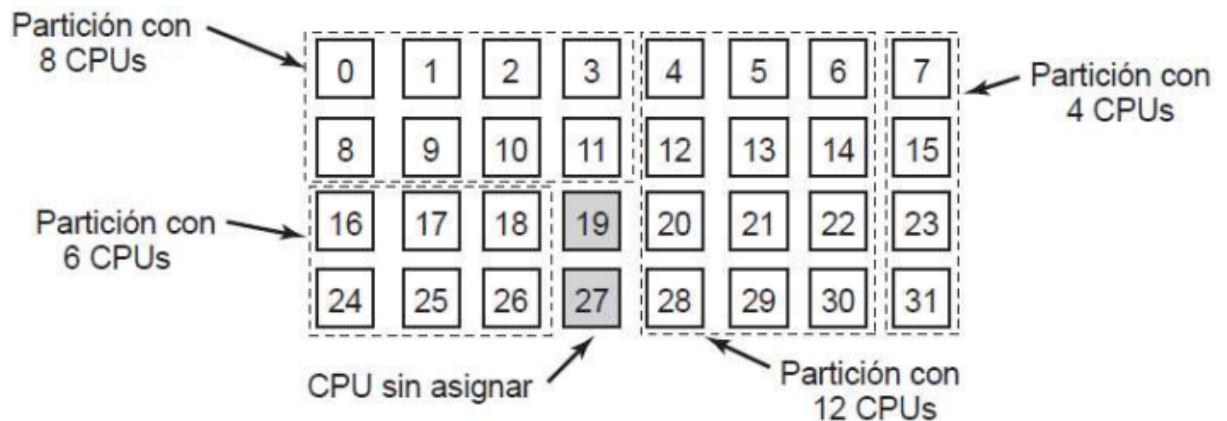
Se planifican en conjunto (en varias CPUs)

Mejora en trabajos en paralelo

El grupo se planifica si hay CPUs libres para cada hilo del grupo

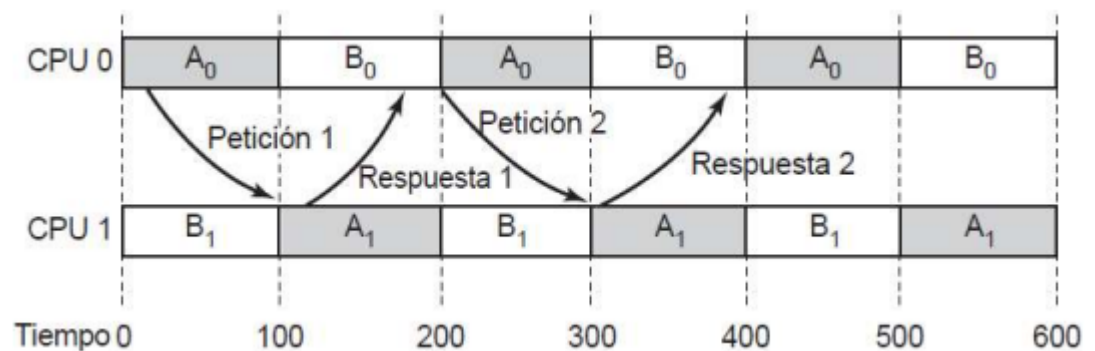
Si no las hay, el grupo espera

No hay multiprogramación por CPU (cada CPU ejecuta solo 1 hilo), baja la productividad



Se podrían multiprogramar las CPU, pero podría ocurrir que los hilos no se ejecuten sincrónicamente, ya que se planifican independientemente

Supongamos una situación de 2 hilos A_0 y A_1 que se ejecutan intercambiando mensajes compartiendo CPU con los hilos de un proceso B:



Como A_0 se ejecuta en un intervalo distinto que A_1 ocurre que el lapso de ejecución de A es de 200 mseg, cuando podría haber sido de 100 si se hubieran ejecutado en el mismo intervalo

Planificación por pandillas (una solución al problema anterior)

Los hilos relacionados se toman como una pandilla

Todos los miembros de una pandilla se ejecutan simultáneamente en distintas CPUs multiprogramadas

Todos los miembros de la pandilla inician y terminan sus intervalos en conjunto

Supongamos un multiprocesador con 6 CPU utilizadas por 5 procesos (A..E) y un total de 24 hilos

En el instante de tiempo 0, se programan los hilos $A_0..A_5$

Luego se ejecuta $B_0, B_1, B_2, C_0, C_1, C_2$, etc...

Se continúa la ejecución cíclicamente

La idea de ejecutar todos los hilos de un proceso juntos, permite que el pasaje de mensajes se realice de manera más rápida y eficiente

Multicomputadoras

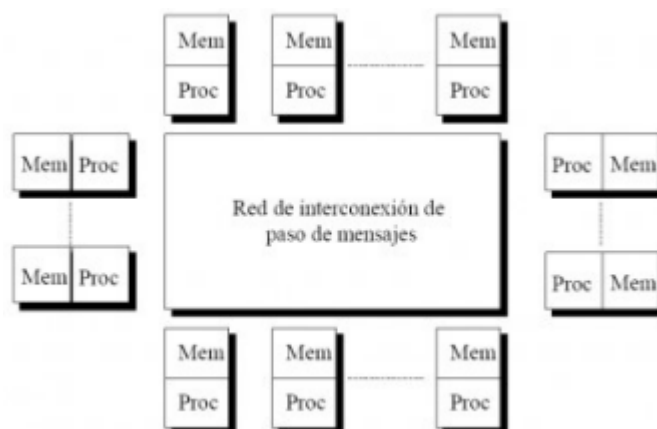
También conocidos como cluster de computadoras

CPUs con acoplamiento fuerte

No se comparte memoria, cada CPU tiene la suya

Multicomputadora:

- PCs con una interfaz de red de alto rendimiento
- Generalmente carecen de placa de video, sonido y en algunos casos disco



Poseen mucha CPU, memoria y placas de interconexión redundantes:

- Red:
 - Ethernet (10, 40, 100, 400 Gbps)
 - Infiniband (120 Gbps)
- HBA:
 - Host Bus Adapter para conexión storage
 - Generalmente fibra óptica y usa el protocolo FibreChannel

Es necesario diseñar correctamente la red

Software de comunicación a nivel de usuario

Para comunicarse, los procesos en distintas CPUs en una multicomputadora se envían mensajes entre sí

Pasaje de mensajes

Send y Receive con y sin bloqueo:

- El SO Provee interfaces a los procesos de usuario para realizar la comunicación
- Envío con bloqueo (síncronas) (la CPU queda inactiva durante la transmisión del mensaje)
- Envío sin bloqueo con copia (asíncronas) (se desperdicia el tiempo de la CPU por la copia adicional)
- Envío sin bloqueo con interrupción (dificulta la programación)

RPC

Para obtener un mayor nivel de abstracción

Permite invocar a procedimientos que se ejecutan en otra CPU

El proceso en una máquina invoca al procedimiento remoto y se bloquea hasta que llegue la respuesta

La comunicación es transparente al programador

Planificación

Cada nodo tiene su propio conjunto de procesos

Un nodo no toma procesos de otros para ejecutarlos, ya que sería bastante costoso

Es importante la asignación de procesos a los nodos

Se puede aplicar el concepto de planificación por pandillas

Sincronización entre los nodos en cada inicio de una ranura de tiempo

Balanceo de carga

Se representa al sistema como un grafo

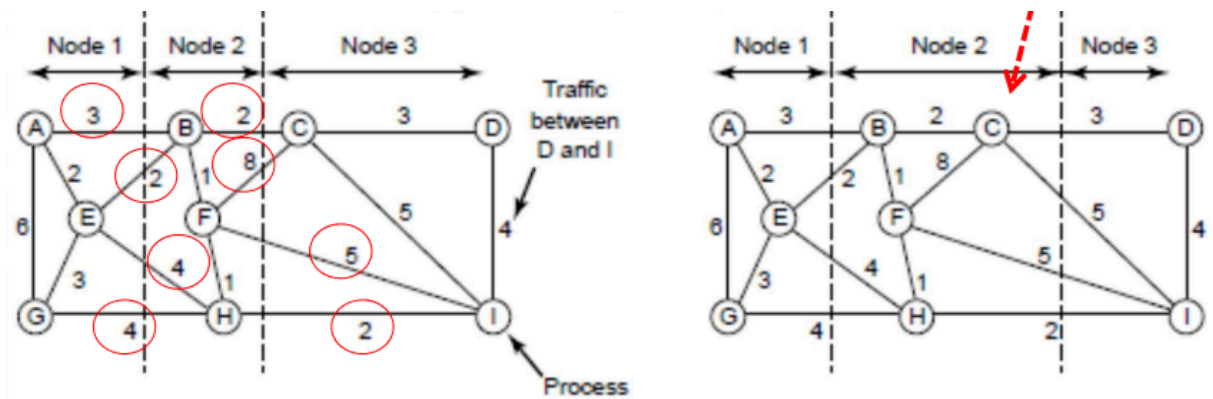
Cada nodo es un proceso, y la comunicación entre ellos se representa a través de una arista

Se debe particionar el grafo en tantos subgrafos como nodos se tengan, teniendo en cuenta:

- Requerimientos totales de CPU
- Requerimientos totales de memoria
- Minimizar la cantidad de aristas entre nodos de distintos subgrafos (tráfico de red)
- Buscar clusters con acoplamiento fuerte

Tenemos 9 procesos, sabiendo el costo de comunicación entre estos y tenemos 3 nodos

Tráfico de la red, suma de los pesos entre enlaces de 2 nodos diferentes (ej1: 30 , ej2: 28)



Sistemas distribuidos

A distributed system is a collection of independent computers that appears to its users as a single coherent system (Tanembaum)

A distributed system is one in which components located at networked computers communicate and coordinate their actions only by passing messages (Colouris)

Similar a las multicomputadoras

Cada nodo tiene su propia memoria privada

Menor acoplamiento

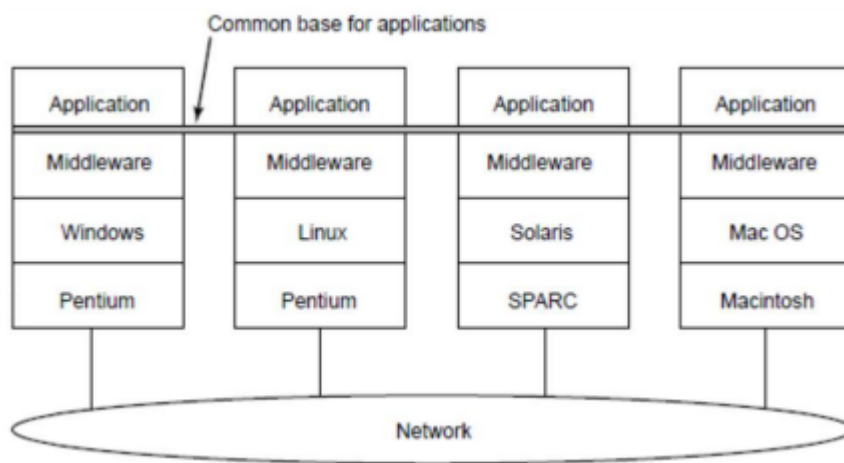
Se pueden encontrar esparcidos por todo el mundo

Cada nodo es una PC completa incluyendo dispositivos

Cada nodo puede ejecutar un SO y Hardware diferente, incluyendo su propio sistemas de archivos

Middleware

Capa de software por encima del SO que permite una uniformidad entre los distintos SOs



El middleware es una capa de Software fundamental en un Sistema distribuido:

- Provee una interfaz común a todos los procesos
- Soluciona los problemas de heterogeneidad
- Provee servicios a las capas superiores
- Provee estructuras de datos y operaciones que permiten a los procesos y usuarios inter-operar, de manera consistente, entre máquinas remotas

Clase 11 - Deadlocks

Definición

Un conjunto de procesos están en deadlock cuando cada uno de ellos está esperando por un recurso que está siendo usado por otro proceso del mismo conjunto

Un estado de Deadlock puede involucrar recursos de diferentes tipos

Ejemplos

Con Procesos:

- Un proceso A pide un scanner
- Un proceso B pide una grabadora de CD
- El proceso A pide ahora la grabadora de CD
- El proceso B quiere el scanner

En una BD:

- Un proceso A bloquea el registro R1
- Un proceso B bloquea el registro R2
- Luego cada proceso trata de bloquear el registro que está usando el otro

Recursos

Todo sistema contiene recursos

Recursos físicos: CPU, memoria, dispositivos

Recursos lógicos: archivos, registros, semáforos, etc.

Recursos apropiativos: se le pueden quitar al proceso sin efectos dañinos (ej: memoria, CPU)

Recursos no apropiativos: si se le saca al proceso, éste falla (interrumpir una escritura a CD, impresora)

Cada recurso R_j puede tener W_j instancias idénticas (puede haber 2 impresoras del mismo tipo). Si son idénticas, se puede asignar cualquier instancia del recurso

Clase de Recursos: es el conjunto de instancias de un recurso

Seguindo un modo de operación normal, un proceso emplea un recurso siguiendo la siguiente secuencia:

1. Solicitud: si no puede ser concedida inmediatamente, el proceso deberá esperar a que el recurso sea liberado
2. Uso: el proceso puede operar sobre el recurso
3. Liberación: se libera el recurso para que pueda ser utilizado por otro proceso

Si el recurso que se quiere utilizar está ocupado, se sigue un “Ciclo corto de solicitud”:

1. Solicitud fallida
2. Espera inactiva
3. Nuevo intento de solicitud

Representación de procesos y recursos

Para poder representar la asignación de recursos, se utiliza un Grafo de Asignación de recursos

El grafo permite visualizar el estado de los recursos del sistema y procesos en un momento determinado

Cada proceso o recurso es representado por un nodo

Un recurso con varias instancias posee más de un “Punto” dentro del nodo

Una arista representa una relación entre un Proceso y un Recurso

Notar que las aristas son dirigidas y dependiendo de la dirección indican distintos estados



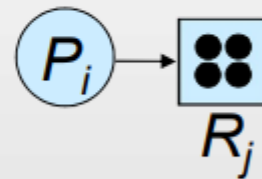
Proceso



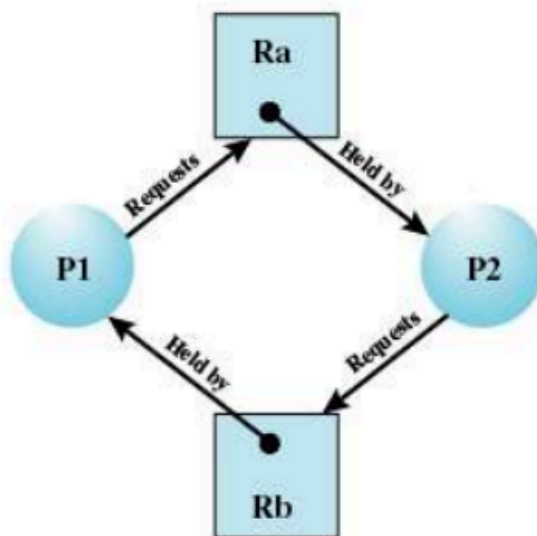
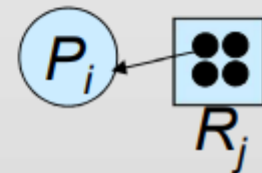
Tipo de Recurso con
4 instancias



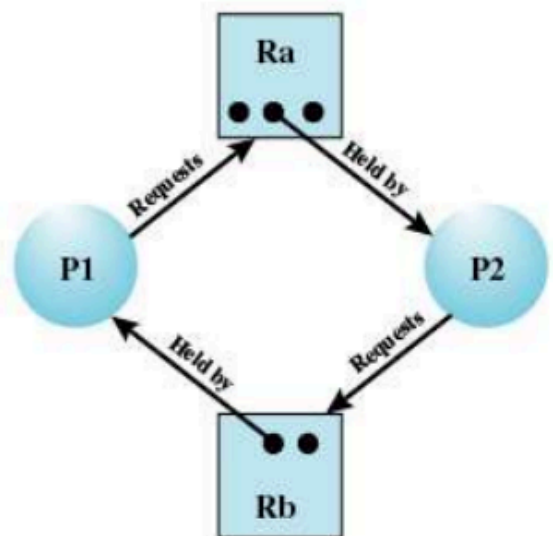
P_i solicita una
instancia de R_j



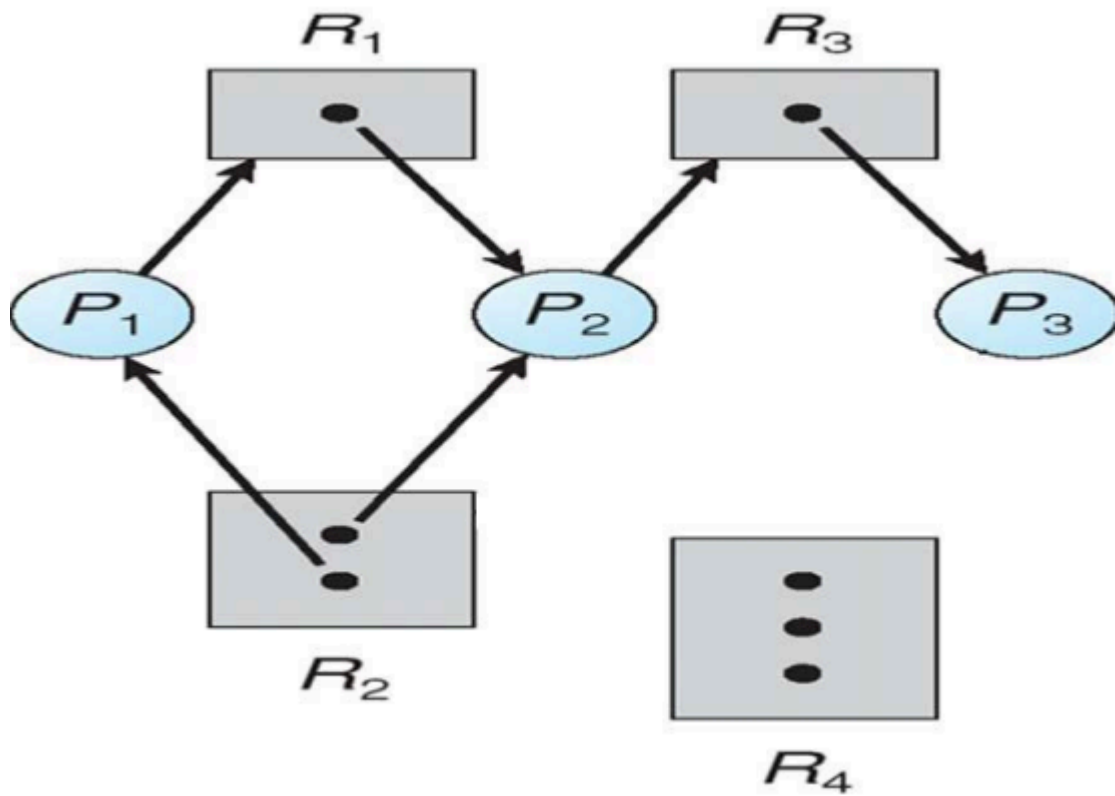
P_i tiene asignada una
instancia de R_j



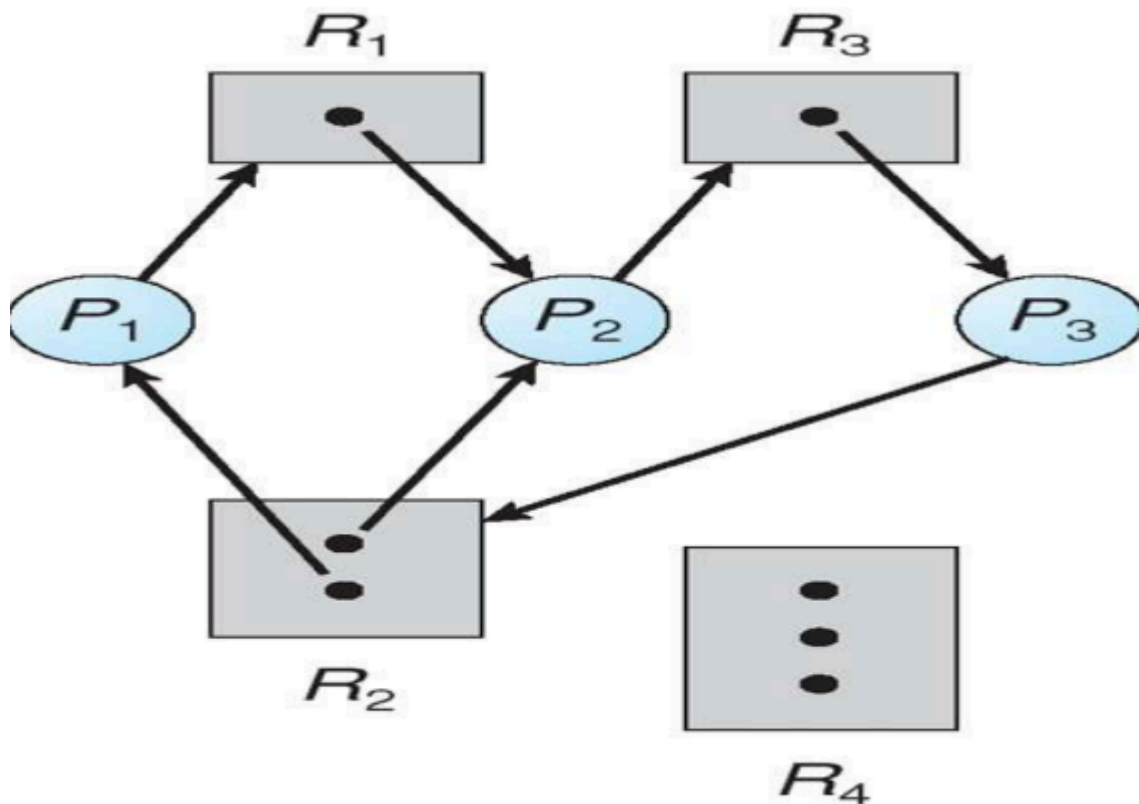
(c) Circular wait



(d) No deadlock



En la última imagen hay un bucle, pero no hay deadlock porque al terminar P_3 , se liberará la instancia de R_3 , por lo que P_2 podrá tomarlo. Sin embargo, este es un estado inseguro ya que, si P_3 no termina nunca, habría deadlock



En este otro caso hay deadlock

Hay deadlock si...

Si el grafo no contiene ciclos → NO hay deadlock

Si el grafo contiene un ciclo:

- Si sólo hay una instancia por tipo de recurso → SI hay deadlock
- Si hay varias instancias por tipo de recurso → hay posibilidad de deadlock

Condiciones para deadlock

1. Exclusión mutua: en un instante de tiempo dado, sólo un proceso puede utilizar una instancia de un recurso
2. Retención y espera: los procesos deben mantener los recursos asignados y esperar por la asignación de los nuevos requeridos
3. No apropiación: los recursos no pueden ser quitados a un proceso que actualmente los posea
4. Espera circular: el proceso forma parte de una lista circular en la que cada proceso de la lista está esperando por al menos un recurso asignado a otro proceso de la lista

Para estar en presencia de un Deadlock se deben cumplir todas!

Métodos para el tratamiento de deadlock

Existen distintos enfoques con el fin de llevar adelante el manejo de Deadlocks:

1. Usar un protocolo que asegure que NUNCA se entrará en estado de deadlock
 - a. Prevenir: que no se cumple alguna de las 4 condiciones
 - b. Evitar: tomar decisiones de asignación en base al estado del sistema
2. Permitir el estado de deadlock y luego recuperar
3. Ignorar el problema y esperar que nunca ocurra un deadlock 🤖

Prevention vs avoidance

Prevention (prevenir la formación del deadlock):

- Que por lo menos una de las condiciones no pueda mantenerse

Avoidance (evitar la formación del deadlock):

- Se imponen restricciones en la forma en que los procesos REQUIEREN los recursos
- Asignar cuidadosamente los recursos, manteniendo información actualizada sobre requerimiento y uso de recursos

Prevenir

Exclusión mutua

Si ningún recurso se asignara de manera exclusiva (no siempre se puede), no habría interbloqueo

Considerar que hay recursos compartibles (archivos read only, memoria) y no compartibles (impresora)

Mantener la exclusión mutua para los no compartibles puede resultar complejo:

- Se pueden implementar esquemas de encolamiento y que el recurso sea manejado por un proceso global (spooler)
- De esta manera no se bloquea el recurso no compartible
- Pero... el spooler podría llenarse y bloquear procesos

Los recursos compartibles NO requieren mantener la exclusión mutua

Retención y espera

Se basa en que si un proceso requiere un recurso que no está disponible, debe liberar otros

Alternativas:

1. Un proceso debe requerir y reservar todos los recursos a usar antes de comenzar la ejecución (precedencia de las system calls que hacen el requerimiento antes de cualquier otra system call)
2. El proceso puede requerir recursos sólo cuando no tiene ninguno (al comienzo de su ejecución generalmente)

Desventajas:

- Baja utilización de recursos
- Posibilidad de inanición de alguno de los procesos (starvation, o espera infinita)

No apropiación

No siempre se puede atacar esta condición, ya que no siempre puede expropiarse un recurso a un proceso

Una posible solución es virtualizar recursos:

- El proceso no accede directamente al recurso, sino que accede a un demonio que lo administra
- Solo el demonio tiene acceso al recurso, y a medida que los procesos requieren el recurso, interactúan con el demonio quien almacena los trabajos en una cola (spooler)
- De esta manera se logra que el recurso físico pueda ser “utilizado” por varios procesos al mismo tiempo (una forma ficticia de compartir el recurso, ya que en realidad solo 1 proceso a la vez lo está utilizando)

Espera circular

Se define un ordenamiento de los recursos. Luego, un proceso puede requerir recursos en un orden numérico ascendente

Sea $F: (R \rightarrow N)$, N conjunto de los naturales

La función F asigna un número único a cada recurso (los números pequeños para recursos muy usados)

Un proceso, que ya tiene R_i puede requerir R_j si y sólo si $F(R_i) < F(R_j)$ (solicita recursos con número mayor a los ya asignados)

Como no se puede dar que $F(R_j) > F(R_i)$ y $F(R_j) < F(R_i)$, podemos garantizar que no habrá un bucle en el grafo de asignación de recursos

Solo se obtienen recursos si se da en orden numérico ascendente

Ejemplo:

- Supongamos que se han definido los siguientes valores: $F(CD)=1$; $F(\text{disco duro})=4$, $F(\text{impresora})=7$

- Un proceso que ya tiene asignado el disco, puede pedir la impresora (pues $F(\text{disco duro}) < F(\text{impresora})$)
- Si ya tiene la impresora, no puede solicitar el CD

Evitar

Requiere que el SO tenga información ANTES sobre el uso de recursos

El SO cuenta con información sobre el uso de los recursos:

- Cómo son requeridos por parte de los procesos
- En qué momento son requeridos
- La demanda máxima, etc.

La técnica se basa en tomar decisiones acerca de la asignación de los recursos, con el fin de que no se llegue a un estado de Deadlock

En todo momento el SO toma decisiones dinámicas acerca de la asignación

Si en algún momento se evalúa que podría entrarse en un estado de Deadlock, la asignación de recursos es denegada

Desventajas en implementación: puede producir una baja utilización de los recursos y de la performance del sistema debido a los cálculos que deben realizarse

Estado seguro e inseguro

Un sistema está en un estado seguro si el SO puede asignar recursos a cada proceso de un conjunto de alguna manera, evitando el deadlock

Cuando un proceso solicita un recurso disponible, el sistema DEBE DECIDIR si la asignación inmediata deja el sistema en un estado seguro

Debe haber una secuencia "cadena segura" de TODOS procesos $\langle P_0, P_1, \dots, P_n \rangle$, que puedan ejecutarse con todos los recursos disponibles sin que haya deadlock

El sistema está en estado seguro si existe secuencia $\langle P_0, P_1, \dots, P_n \rangle$ (cadena segura) para TODOS los procesos del sistema

Si existe una secuencia, significa que se pueden satisfacer los requerimientos, teniendo en cuenta lo actualmente asignado a los procesos en ejecución

Si los recursos necesarios de P_i no están disponibles inmediatamente, entonces P_i puede esperar hasta que todos los P_j hayan terminado

Cuando P_j está terminado, P_i puede obtener los recursos necesarios, ejecutar, devolver los recursos asignados y finalizar

Cuando P_i termina, $P_i + 1$ puede obtener sus recursos necesarios, y así sucesivamente

Si no se puede construir esta secuencia, el estado del sistema es inseguro

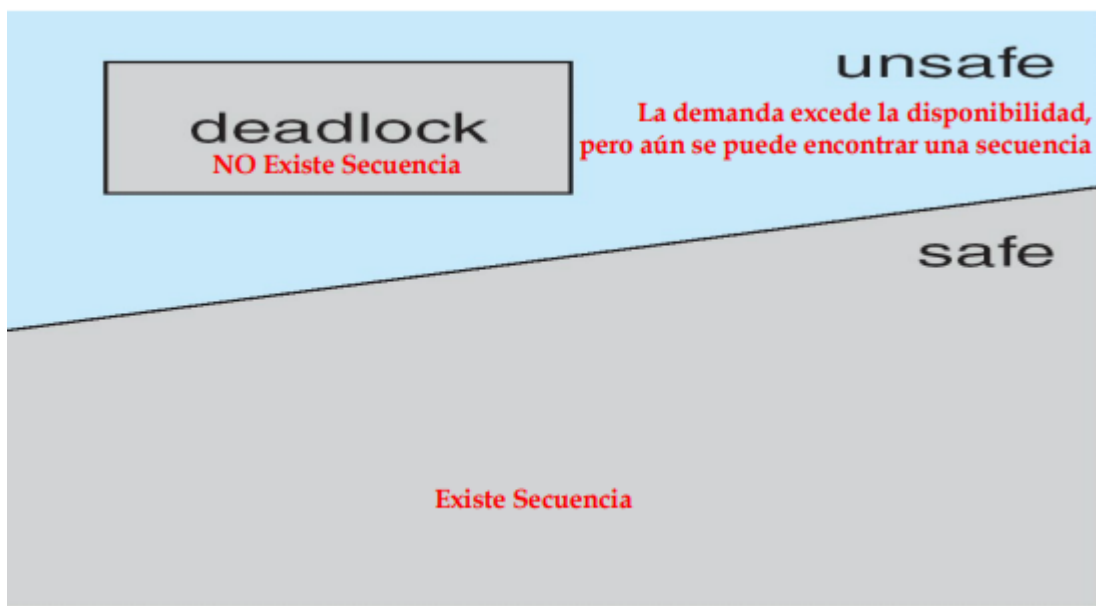
Un estado seguro garantiza que NO hay deadlock

En estado inseguro hay posibilidad de que haya deadlock

No todo estado inseguro es deadlock

Puede ocurrir que no exista cadena de asignación temporalmente, pero ante la terminación de un proceso, se liberen recursos y se pueda construir la cadena

Si hay deadlock, estoy en estado inseguro



Algoritmos para evitar el deadlock

Instancia única de un tipo de recurso:

- Algoritmo que determina el estado seguro de un sistema
- Utilizar un grafo de asignación de recursos
- Se debe encontrar una secuencia segura (evitar los bucles)

Múltiples instancias de un tipo de recurso:

- Se utiliza el algoritmo del banquero

- Algoritmo teórico
- Busca encontrar una secuencia segura de asignación

Algoritmo del banquero

Se aplica para sistemas con múltiples instancias de cada recurso

Los procesos declaran el número máximo de instancias de cada recurso que necesitaría

Ese número no puede exceder el total de instancias de recursos de ese tipo en el sistema

El SO decidirá en qué momento asignarlos, garantizando un estado seguro

Estructuras asociadas:

- n : cantidad de procesos
- m : cantidad de tipos de recursos
- disponible:
 - Vector de m componentes, con la cantidad de recursos disponibles para cada tipo
 - Si $disponible[j]=k$, indica que hay k instancias del recurso R_j
- asignación:
 - Matriz de $n \times m$ que indica cuantos recursos tiene asignados cada proceso
 - $asignacion(i, j)=k$, indica que hay k instancias del recurso R_j asignadas a P_i
- max :
 - Matriz de $n \times m$ que indica la cantidad máxima de recursos que un proceso necesitará
 - $Max(i, j)=k$, indica que P_i necesitará en total k instancias del recurso R_j
- need:
 - Matriz de $n \times m$ que indica la cantidad de recursos que le faltan a un proceso para completar su ejecución ($need = max - asignación$)
 - $Need(i, j)=k$, indica que P_i necesitará k instancias más de las que ya tiene, del recurso R_j

Si X e Y son vectores de n componentes, decimos que $X \leq Y$ si y sólo si $X(i) \leq Y(i)$, para todo $i=1, \dots, n$

Para este algoritmo, tomaremos filas de las matrices como si fueran vectores

Recursos asignados a P_i representados por el vector asignacion $_i$ que es la fila i de la matriz asignacion

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Max

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Asignacion

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

Necesidad

R1	R2	R3
0	1	1

Vector de disponibles D

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Max

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Asignacion

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

Necesidad

R1	R2	R3
0	1	1

Vector de disponibles D

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Max

	R1	R2	R3
P1	1	0	0
P2	6	1	3
P3	2	1	1
P4	0	0	2

Asignacion

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

Necesidad

R1	R2	R3
0	1	0

Vector de disponibles D

(continuación)

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Max

	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Asignacion

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

Necesidad

R1	R2	R3
6	2	3

Vector de disponibles D

□ El algoritmo continúa con el fin de evaluar si existe una secuencia segura. En tal caso, podemos decir que el sistema se encuentra en un estado seguro

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Max

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	0

Asignacion

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	0

Necesidad

R1	R2	R3
9	3	6

Vector de disponibles D

Detección y recuperación

Permitir el estado de deadlock y luego recuperar

La técnica determina si ocurrió un deadlock, y en tal caso busca recuperarse del mismo

Para ello vamos a contar con:

1. Algoritmo que examine si ocurrió un deadlock
2. Algoritmo para recuperación del deadlock

Detección

Con recursos con una sola instancia: análisis del grafo de asignación

Con recursos de varias instancias: algoritmo del Banquero

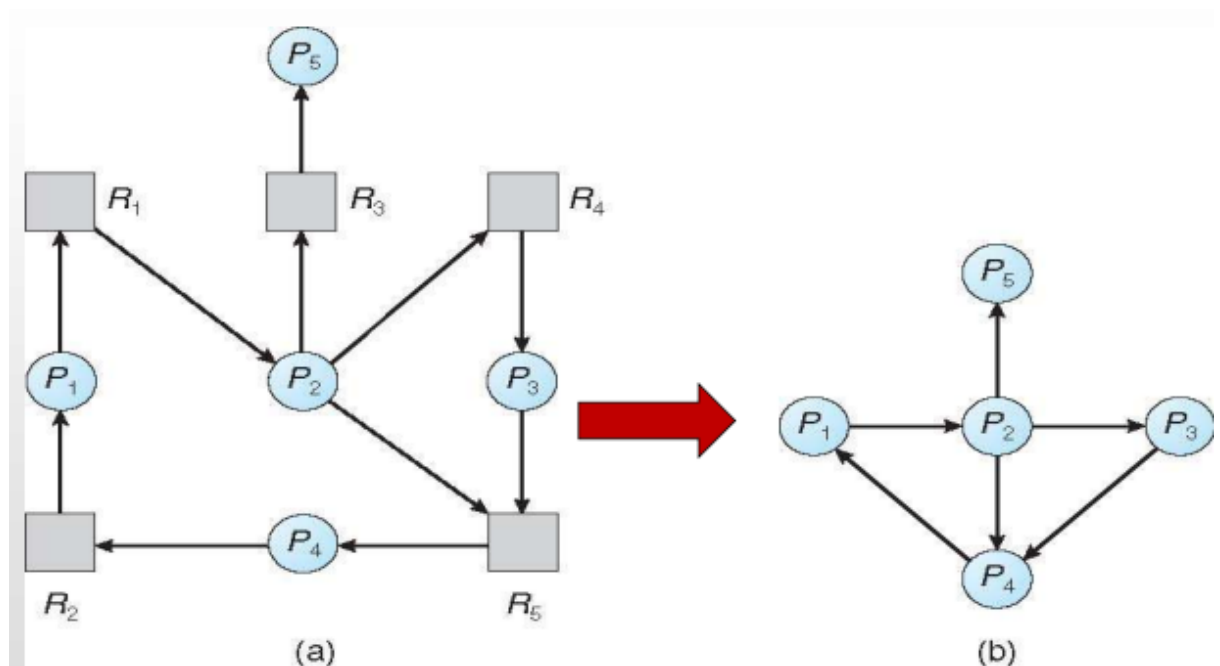
Instancia única

Mantener un grafo wait-for (Grafo de espera):

- Nodos = Procesos
- $P_i \rightarrow P_j$, si P_i espera que P_j libere recurso R_q

El algoritmo se basa en buscar periódicamente un ciclo en el grafo

Si hay un ciclo, existe un bloqueo



¿Cuándo uso el algoritmo de detección?

Cuándo, y con qué frecuencia, invocar depende de:

- ¿Con qué frecuencia es probable que ocurra un deadlock?
- ¿Cuántos procesos tendrán que ser revertidos (rolledback)?

La frecuencia con la que se debe ejecutar el algoritmo de detección de interbloqueos, suele ser un parámetro del sistema

Los motores de BD también poseen algoritmos de detección de deadlocks

Por ejemplo SQL server lo corre cada 5 segundos

Una posibilidad extrema es comprobar el estado cada vez que se solicita un recurso y estos no pueden ser asignados → consume mucha CPU

Si el algoritmo de detección se invoca arbitrariamente, puede haber muchos ciclos en el grafo de recursos y, por lo tanto, no podríamos decir cuál de los muchos procesos bloqueados "causó" el bloqueo → hay que encontrar un término medio

Si se pasa mucho tiempo sin invocar el algoritmo de detección podría haber muchos procesos en estado de deadlock y realizar la recuperación (el rollback), podría ser muy costosa → hay que encontrar un término medio

Otra opción es ejecutar cuando la utilización del procesador desciende su actividad por debajo de un determinado porcentaje de utilización (recordar que un 'deadlock' eventualmente disminuye el uso de procesos en actividad)

Recuperación

Cuando un algoritmo de detección determina que existe un deadlock, existen varias alternativas para tratar de eliminarlo:

1. El Operador lo puede resolver manualmente (informar al operador del SO)
2. Esperar que el sistema se recupere automáticamente del deadlock. El Sistema rompe el deadlock

En cualquiera de los casos, las alternativas son:

- Abortar 1 o más procesos para romper ciclo
- Expropiar recursos a 1 o más procesos del ciclo

Matar procesos

Para eliminar el deadlock matando procesos pueden usarse 2 métodos:

- Matar todos los procesos en estado de deadlock. Es simple pero a un muy alto costo
- Matar de a un proceso por vez hasta eliminar el ciclo:
 - Considerable overhead ya que por cada proceso que vamos eliminando se debe ejecutar nuevamente el algoritmo de detección para verificar si el deadlock efectivamente desapareció o no

Terminación de procesos:

- Puede no ser fácil terminar un proceso (puede estar actualizando un archivo)
- Previo a terminar el proceso hay que hacerlo llegar a un estado seguro (check point) de modo que al reanudarlo quede consistente
- Se presenta un nuevo problema de política o decisión: selección de la víctima
- Se debe seleccionar aquel proceso cuya terminación represente el costo mínimo para el sistema
- Hay que asegurarse de no seleccionar siempre al mismo proceso (inanición)

¿En qué orden debemos optar por abortar/terminar?

- Prioridad más baja
- Menor cantidad de tiempo de CPU hasta el momento
- Mayor tiempo restante estimado para terminar
- Menor cantidad de recursos asignados hasta ahora
- Tiempo faltante para la liberación de recursos asignados
- ¿Cuántos procesos tendrán que ser terminados?
- ¿El proceso es interactivo o batch?
- ¿Puede volver atrás?

Ideal: elegir un proceso que se pueda volver a ejecutar sin problemas (ej. una compilación)

Resumen de detección y recuperación

La detección y recuperación de deadlocks proporciona un mayor grado potencial de concurrencia que las técnicas de prevención o de evitación

Hay sobrecarga en tiempo de ejecución de la detección

Hay procesos que son reiniciados o rollback lo que genera demoras en la ejecución

La detección y recuperación de deadlocks puede ser atractiva en sistemas con una baja probabilidad de deadlocks

En sistemas con elevada carga, se sabe que la concesión sin restricciones de peticiones de recursos puede conducir a frecuentes deadlocks

Estrategia combinada para el manejo de deadlocks

Ninguno de los métodos presentados es 100% adecuado para ser utilizado como estrategia exclusiva de manejo de deadlocks en un sistema complejo

La prevención, evitación y detección pueden combinarse para obtener una máxima efectividad

Se dividen los recursos del sistema en “clases de recursos”:

- Para cada clase se aplica el método más adecuado de manejo de interbloqueo
- Depende de si soportan apropiaciones
- Depende de si se puede predecir el comportamiento del recurso